SymSan: Mitigating Malware Embedding in Large Language Models via Parameter Space Symmetry

Ming Tan
Information Engineering University, Zhengzhou, China
He'nan Province Key Laboratory of Cyberspace Endogenous Security & Safety, Zhengzhou, China
Key Laboratory of Cyberspace Security Ministry of Education of China, Zhengzhou, China
t4nming@163.com

Wei Li
State Key Laboratory of Integrated Service Networks, Xi'an, China
School of Cyber Engineering, Xidian University, Xi'an, China
wei.li@stu.xidian.edu.cn

Tao Hu*
Information Engineering University, Zhengzhou, China
He'nan Province Key Laboratory of Cyberspace Endogenous Security & Safety, Zhengzhou, China
Key Laboratory of Cyberspace Security Ministry of Education of China, Zhengzhou, China
hutaondsc@163.com

Qian Chen
State Key Laboratory of Integrated Service Networks, Xi'an, China
Institute of Mathematics and Interdisciplinary Sciences, Xidian University, Xi'an, China
chenqian02@xidian.edu.cn

Aodi Liu
Information Engineering University, Zhengzhou, China
He'nan Province Key Laboratory of Information Security, Zhengzhou, China
Northern Institute of Information Technology, Beijing, China
ladyexue@163.com

Hailong Ma*
Information Engineering University, Zhengzhou, China
He'nan Province Key Laboratory of Cyberspace Endogenous Security & Safety, Zhengzhou, China
Key Laboratory of Cyberspace Security Ministry of Education of China, Zhengzhou, China
longmanclear@163.com

Abstract

Despite the considerable advantages of open-source large language models (LLMs) in terms of interpretability and customizability, the full-access sharing paradigm inevitably exposes them to a severe security threat, i.e., malware embedding attacks (MEAs), where adversaries embed malware binaries (i.e., payloads) into model parameters for subsequent execution. While existing sanitization defenses have been explored to eliminate payloads within parameters, they exhibit limited generalization against emerging MEAs and lack theoretical guarantees for model utility preservation. In this paper, we propose SymSan, an effective LLM sanitization method grounded in parameter space symmetry theory. SymSan leverages rigorously designed permutation and scaling symmetries, systematically transforming parameter matrices across complementary component pairs, thereby eliminating embedded payloads while ensuring mathematical functional equivalence. Experiments across diverse LLMs against a wide spectrum of MEAs demonstrate that SymSan achieves at least 0.47 BER against all evaluated existing MEAs while maintaining equivalent model performance on both foundational and downstream tasks, significantly outperforming all baselines.

CCS Concepts

• **Security and privacy** → **Software security engineering**; **Malware and its mitigation**; • **Computing methodologies** → **Machine learning**.

Keywords

Malware embedding attacks, sanitization, open-source large language models, parameter space symmetry

ACM Reference Format:
Ming Tan, Wei Li, Tao Hu, Qian Chen, Aodi Liu, and Hailong Ma. 2026. SymSan: Mitigating Malware Embedding in Large Language Models via Parameter Space Symmetry. In *Proceedings of the 2026 ACM SIGSAC Conference on Computer and Communications Security (CCS '26), November 15–19, 2026, The Hague, Netherlands.* ACM, New York, NY, USA, 15 pages. https://doi.org/10.1145/3830454.3846529

1 Introduction

The rapid evolution of large language models (LLMs) has profoundly advanced artificial intelligence (AI) [26, 45], driving a new wave of innovation across intelligent chatbots [18], vulnerability detection [20], and code analysis [7]. As a compelling alternative, open-source LLMs, i.e., LLMs with transparent codes and model parameters, achieve noteworthy advantages in terms of interpretability [24, 50], customizability [56], and data privacy [28].

The transparency of open-source LLMs is conventionally associated with full access to source codes and model parameters.

*Corresponding authors.



This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.
CCS '26, The Hague, Netherlands.
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2871-6/2026/11
https://doi.org/10.1145/3830454.3846529

However, is the full-access paradigm real transparency? The answer remains negative, as latent vulnerabilities in the source code and the interpretability of model parameters are unrevealed from the very beginning. Consequently, such non-transparency inevitably exposes open-source LLMs to a severe security threat, i.e., malware embedding attacks (MEAs) [4, 22, 34]. To be more specific, MEAs weaponize such non-transparency, where adversaries with full access to model could embed malware binaries (i.e., payloads) into model parameters without detection [8, 13, 14, 16, 22, 41, 42, 48], and subsequently activate them during model deployment [4, 34, 54, 58], thereby compromising the system.

Existing defenses against MEAs primarily fall into two categories: detection and sanitization. While detection methods typically construct classifiers based on parameter statistics to identify compromised models [8, 10, 55], they exhibit limited efficacy and poor generalization, as classifiers are costly to train and quickly fail against emerging attacks, e.g., MaleficNet [13, 14]. By contrast, sanitization proactively eliminates embedded payloads through parameter perturbation [1, 6], delivering improved efficacy against unknown MEAs while avoiding prohibitive training overhead. In this regard, parameter randomization (e.g., Gaussian Noise [1] and CDR [6]), model compression (e.g., pruning [13, 14] and quantization [34]), fine-tuning [34, 48], and NeuPerm [9] were investigated.

However, thus far, the prior art of sanitization defenses have been confined to conventional deep neural networks (DNNs), and manifest several drawbacks. (1) The defense efficacy suffers severely from limited generalization against emerging attacks. To date, no existing method, including model compression and fine-tuning, can effectively defend against advanced robust-encoding attacks such as MEASER [34] and FREEZER [48]. (2) Existing defenses struggle to preserve model utility owing to insufficient theoretical guarantees. The parameter perturbations they introduce (e.g., via fine-tuning) empirically degrade model performance [5, 29, 53], leaving post-defense model utility difficult to predict or explain. (3) Furthermore, the complexity of modern LLMs exacerbates these issues, as their massive parameter space poses significant challenges to comprehensive defense coverage, and the intricate computation mechanism amplifies perturbation-induced utility degradation [2, 38]. To sum up, the inherent drawbacks of current methods call for a systematic exploration of sounder defenses. As a result, an intractable question springs to mind:

How to sanitize LLMs effectively while theoretically preserving model utility? Notably, answering this question is far from trivial, as the complexity and diversity of MEA strategies hinder the design of a one-size-fits-all sanitization across all attack types. More importantly, establishing rigorous guarantees of functional preservation across billions of parameters in complex LLM architectures remains a formidable challenge.

Our response to this question is SymSan, an effective sanitization for LLMs grounded in parameter space symmetry theory. The key insight behind SymSan stems from a fundamental observation: MEAs critically rely on both parameter indices and numerical values to embed and recover payloads. Inspired by functionally equivalent transformations from Wu's endogenous safety and security [43, 44], SymSan employs permutation and scaling symmetries to shuffle the indices and values respectively, preventing malicious payload recovery while keeping functionality, and achieves endogenous security of model parameters. To ensure comprehensive sanitization across all model parameters, SymSan systematically decomposes LLM architectures into a principled set of complementary component pairs. Within each pair, SymSan applies complementary permutation or scaling symmetry transformations to their respective parameters while strictly preserving functional equivalence. By aggregating all pair-wise transformations, each parameter matrix ultimately undergoes a composite transformation, thoroughly shuffling both indices and values. Consequently, all payloads embedded across the entire model are eliminated, while model functionality remains strictly invariant.

The main contributions are summarized as follows:

- We propose SymSan, the first principled LLM sanitization method that jointly leverages permutation and scaling symmetries to shuffle both parameter indices and numerical values, effectively eliminating embedded malware payloads against MEAs.
- We design a functional equivalence transformation framework, which decomposes an LLM into paired components for complementary symmetry transformations. By aggregating all pairwise transformations, SymSan achieves provable functional equivalence across the entire model.
- We conduct extensive evaluations across diverse LLMs against a wide spectrum of MEAs. Empirical results show that SymSan achieves at least 0.47 BER against existing MEAs, while maintaining performance comparable to undefended models on both foundational and downstream tasks, significantly outperforming all baselines.

2 Related Work

2.1 Malware embedding attack and defense

While open-source LLMs have made AI capabilities widely accessible [28], the uninterpretable nature of their parameters and the full-access sharing paradigm have introduced a severe security threat, known as malware embedding attacks (MEAs). Loosely speaking, MEAs stealthily embed malware binaries (i.e., payloads) within model parameters before model sharing to evade traditional security scanners [8, 13, 14, 16, 22, 41, 42, 48], and subsequently recover them for execution during deployment on a target host, thereby compromising the system [4, 34, 54, 58].

Heretofore, a diverse spectrum of MEA techniques have been developed. Liu et al. [22] pioneered MEAs by introducing Resilience Training and Value/Sign Mapping, with the former embedding payloads during model training, and the latter matching payload bits with parameters' value/sign. To further increase embedding capacity and reduce computational costs, Wang et al. [41, 42] proposed MSB and LSB substitution, which directly overwrite the most and least significant bits of parameters with payload bits, respectively. Building upon this, MalModel [16] advances LSB substitution by selecting specific layers for embedding to optimize stealthiness. Nevertheless, since aforementioned embeddings are easily thwarted by model compression and fine-tuning, the latest paradigm has shifted toward robust encoding and redundancy techniques. Along this line, MaleficNet [13, 14] and MEASER [34] integrate spread-spectrum modulation with LDPC error correction to withstand such parameter modifications, whereas FREEZER [48] employs exponent-bit

Table 1: A summary of the existing defenses evaluated in this paper. "Equivalence" indicates whether the method theoretically preserves model functional equivalence. "Universality" reflects the ability to eliminate payloads across all model parameters rather than specific components. "Generalization" and "Transferability" signify whether the defense is effective across all existing MEAs and model architectures, respectively. "Training-Free" and "Data-Free" denote that a method is independent of training and datasets, respectively. Symbols: ● (Yes), ◑ (Partial), and ○ (No).

Defense	Equivalence	Universality	Generalization	Transferability	Training-Free	Data-Free
SymSan	●	●	●	●	●	●
NeuPerm [9]	●	○	○	○	●	●
Noise [1]	○	◑	○	●	●	●
CDR [6]	○	◑	○	●	●	●
FT [34, 48]	○	◑	○	●	○	○
Prune [13, 14]	○	○	○	●	●	◑
Quant. [34]	○	○	○	●	●	◑

embedding alongside dual-majority voting to substantially enhance embedding robustness.

At the opposite end of the spectrum, countermeasures were proposed to mitigate MEAs. Existing detection methods attempt to identify compromised parameters by constructing statistical classifiers [8, 10, 55], yet they exhibit limited efficacy and fail to generalize beyond specific attacks. Concurrently, prevention (sanitization) methods seek to eliminate payloads by perturbing parameters. For instance, Amit et al. [1] add Gaussian noise to the parameters to actively corrupt concealed payloads, while Dubin et al. [6] employ random bit substitution to overwrite the least significant bits. Additionally, model compression techniques, such as pruning [13, 14] and quantization [34], implicitly destroy embedded payloads by discarding parameter subsets or restricting numerical precision. Fine-tuning [34, 48] is also widely adopted to overwrite compromised parameters through retraining on additional data. Nevertheless, despite their elimination efficacy, these approaches inevitably impose a severe trade-off, either degrading the utility of the LLMs or incurring substantial computational overhead. To circumvent this trade-off, NeuPerm [9] leverages permutation symmetry to eliminate payloads without compromising model utility, yet it is restricted to specific network components and leaves the remaining parameters unprotected. By contrast, SymSan leverages parameter space symmetry to universally eliminate payloads hidden across all model parameters, with theoretically no utility degradation. Table 1 compares the capabilities of these sanitization methods.

2.2 Parameter space symmetry

The overparameterization of modern deep neural networks naturally gives rise to parameter space symmetry [19, 32, 52]. In the strictest form, a parameter space symmetry is a transformation $g \cdot \theta$ of the network parameters θ that preserve the functionality of model f for all input $\mathbf{x}$ [52], defined as $f_{g\cdot\theta}(\mathbf{x}) = f_\theta(\mathbf{x})$, where g is an invertible action from a symmetry group G.

Existing research has systematically unveiled a rich taxonomy of symmetries across diverse network components, advancing from permutation symmetry in feed-forward networks (FFNs) [11] to complex forms in modern architectures [49]. Notably, Transformers [38] are naturally endowed with permutation, rotation, and scaling symmetries [9, 49].

Driven by these symmetry findings, the practical applications of such symmetries have garnered considerable attention, primarily focusing on the context of loss landscapes [36, 46], optimization dynamics [25], and mode connectivity [51]. However, its profound promise for enhancing model security remains severely underexplored. Heretofore, while a handful of pioneering studies have attempted to leverage it for mitigating backdoor attacks [30, 47] or enhancing adversarial robustness [40], its potential for countering MEAs remains virtually untapped. Given that MEAs inherently rely on the precise manipulation of parameters' bits to conceal payloads, we argue that the parameter transformations induced by parameter space symmetry fundamentally eliminate the embedded payloads without compromising model utility.

Motivated by this, SymSan leverages parameter space symmetry to establish a systematic defense mechanism against MEAs. So far as we know, SymSan is the first symmetry-based framework to simultaneously achieve theoretically lossless utility and full-parameter payload elimination.

3 System Model, Threat Model and Design Goal

In this section, we formalize the system model, define the threat model, and identify our design goal.

3.1 System model

Without loss of generality, we define an open-source LLM as a probabilistic function $F_\Theta : \mathcal{X} \to \mathcal{P}(\mathcal{Y})$, which, parameterized by Θ, maps an input token sequence $\mathbf{x} \in \mathcal{X}$ to a probability distribution over the output token space $\mathcal{Y}$. Specifically, mainstream transformer-based LLMs consist of numerous components [38] and are parameterized by a group of weight matrices, defined as:

$$\Theta = \left\{\mathbf{W}_{\text{emb}}, \mathbf{W}_{Q/K/V/O}, \mathbf{W}_{g/u/d}, \mathbf{W}_{\text{lm}}, \boldsymbol{\gamma}_{\text{input/post/final}}\right\}, \quad (1)$$

where $\mathbf{W}_{\text{emb}}$ and $\mathbf{W}_{\text{lm}}$ denote the token embedding matrix and the language modeling head, respectively; $\mathbf{W}_{Q/K/V/O}$ correspond to the query, key, value, and output projection weights in multi-head attention (MHA) blocks; $\mathbf{W}_{g/u/d}$ represent the gate, up, and down projection matrices in the feed-forward networks (FFNs); and $\boldsymbol{\gamma}_{\text{input/post/final}}$ are the vectors of the pre-attention, post-attention, and final normalization layers. Specialized architectures, e.g., Mixture-of-Experts (MoE) or multimodal models, may introduce additional components, yet this does not compromise the generality of Eq. (1).

The evolution and utilization of Θ follow a structured lifecycle of three phases: *Training*, *Sharing*, and *Deployment*.

- **Training Phase:** The parameter Θ is optimized by minimizing a loss function $\mathcal{L}$ over a large training corpus $\mathcal{D}$:

$$\Theta^* = \arg\min_\Theta \mathcal{L}(\mathcal{D}, \Theta), \quad (2)$$

where Θ^* refers to the parameters of the well-trained model.

• **Sharing Phase:** To store and transfer the trained parameter matrices, Θ^* is converted into a binary artifact $\mathcal{M}$ by a serialization function $\mathcal{S}(\cdot)$:

$$\mathcal{M} = \mathcal{S}(\Theta^*). \tag{3}$$

Subsequently, the resulting artifact $\mathcal{M}$ is then disseminated via public repositories (e.g., Hugging Face), becoming accessible to the broader community.

• **Deployment Phase:** To deploy and utilize the model, $\mathcal{M}$ is downloaded from the repository by the end-user $\mathcal{U}$, and $\hat{\Theta}$ is reconstructed locally through the deserialization function $\mathcal{S}^{-1}$:

$$\hat{\Theta} = \mathcal{S}^{-1}(\mathcal{M}). \tag{4}$$

In a benign environment, the reconstructed parameters remain identical to the original, i.e., $\hat{\Theta} \equiv \Theta^*$, and are subsequently integrated into downstream applications for inference or further fine-tuning[1].

3.2 Threat model

In the context of an untrusted lifecycle, the end-user $\mathcal{U}$ is vulnerable during the *deployment phase*. In other words, the downloaded artifact $\mathcal{M}$ may conceal malicious code, intended to compromise $\mathcal{U}$ upon local deployment. Specifically, inspired by Liu *et al.* [22], Digregorio *et al.* [4], and Tan *et al.* [34], we consider threat models where the adversary $\mathcal{A}$ embeds malicious binary codes (i.e., payloads) into parameters during *training phase* or *sharing phase*, and activates these concealed binaries via a parameter-independent code (i.e., trigger) to compromise $\mathcal{U}$ during *deployment phase*.

To realize this threat, we assume $\mathcal{A}$ maintains full control over the *training phase* and *sharing phase*, allowing for the manipulation of parameters Θ^* to embed payloads while preserving its utility. Furthermore, we posit that $\mathcal{A}$ can successfully distribute the compromised $\mathcal{M}$ to $\mathcal{U}$ and pre-position the trigger in $\mathcal{U}$'s environment. Crucially, this trigger is designed to remain undetectable until it detects the parameter reconstruction process from $\mathcal{M}$, at which point it executes the hidden payloads within parameters.

As a result, we identify two primary objectives for $\mathcal{A}$: (1) *Effectiveness*, ensuring the payload is successfully recovered and executed upon triggering; and (2) *Stealthiness*, ensuring the embedded payload remains undetectable to malware scanners while preserving the parameters' original performance.

Given the high stealthiness of such attacks, the end-user $\mathcal{U}$ cannot distinguish between benign and compromised artifacts. This necessitates a zero-trust assumption, where $\mathcal{U}$ treats every downloaded $\mathcal{M}$ as potentially malicious. In this work, we focus specifically on parameter-based threats rather than trigger-based attacks, where the latter exploit vulnerabilities in artifact formats and have been widely investigated in prior work [4, 23, 58]. In stark contrast, parameter-based threats exploit inherently uninterpretable model parameters, rendering them far stealthier and undetectable by existing detection mechanisms [8, 10, 55]. We also mention that this threat model shares similarities with backdoor attacks [39], as both utilize triggers to activate hidden behaviors. However, unlike backdoors that focus on manipulating parameters functionality, our approach exploits the parameters as a carrier to directly compromise the host system, posing a far more severe security threat for the open-source model ecosystem.

[1]We abuse $\hat{\Theta}$ and Θ in the rest of this paper, but we mention that both the payload extraction and the defense mechanisms exclusively target the downloaded and reconstructed parameters.

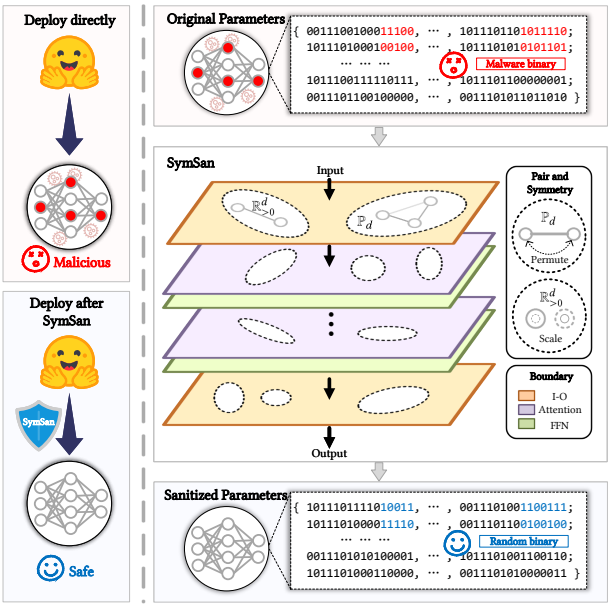


Figure 1: High-level overview of SymSan.

3.3 Design goal

In this paper, we aim to design an effective defense method to safeguard the end-user $\mathcal{U}$ during the *deployment phase*. In a nutshell, our goal is to proactively sanitize the reconstructed parameters $\hat{\Theta}$, eliminating malicious payloads before they can be activated. To this end, two objectives should be achieved:

• *Effectiveness of payloads elimination.* The primary objective is to thoroughly eradicate potentially concealed payloads within $\hat{\Theta}$. The challenges of this objective are twofold. On the one hand, the specific embedding strategy (e.g., LSB substitution or mapping-based embedding) and embedding location (e.g., $\mathbf{W}_Q$ or $\mathbf{W}_g$) is unknown to the defender, necessitating a generalized sanitization approach to counter diverse attacks. On the other hand, target model architectures vary drastically across different types and scales (e.g., Transformer or FFNs), requiring the defense to be sufficiently adaptable to perform comprehensive sanitization on mainstream architectures.

• *Functional equivalence of sanitized parameters.* Functional equivalence is of vital importance, and two issues are highlighted. The underlying issue is maintaining the parameters' foundational ability, meaning the sanitized parameters must retain their original linguistic and reasoning capabilities across general/original tasks. Another crucial issue is ensuring the parameters' adaptability for downstream tasks customization, which guarantees that the sanitized parameters retains its efficacy even after downstream task fine-tuning or model compression for efficient deployment.

4 Methodology

In this section, we propose SymSan to sanitize LLMs while strictly preserving their functional equivalence.

Table 2: Summary of boundaries, complementary pairs, and their corresponding transformations.

Boundary	Pairwise Transformation			Composite Transformation		
	Symmetry Transform	Symmetry Group	Proof	$\mathbf{W}_i$	$\mathbf{W}_i'$	Symmetry Group
I-O Boundary	$\mathbf{P}_1\mathbf{W}_{emb}, \mathbf{P}_1\boldsymbol{\gamma}_{final}, \mathbf{W}_{lm}\mathbf{P}_1^\top$	$\mathbb{P}_d$	Lemma A.1.1	$\mathbf{W}_{emb}$	$\mathbf{P}_1\mathbf{W}_{emb}$	$\mathbb{P}_d$
	$\mathbf{C}_1\boldsymbol{\gamma}_{final}, \mathbf{W}_{lm}\mathbf{C}_1^{-1}$	$\mathbb{R}^d_{>0}$	Lemma A.1.2	$\boldsymbol{\gamma}_{final}$	$\mathbf{C}_1\mathbf{P}_1\boldsymbol{\gamma}_{final}$	$\mathbb{P}_d \times \mathbb{R}^d_{>0}$
				$\mathbf{W}_{lm}$	$\mathbf{W}_{lm}\mathbf{P}_1^\top\mathbf{C}_1^{-1}$	$\mathbb{P}_d \times \mathbb{R}^d_{>0}$
Attention Boundary	$\mathbf{P}_1\boldsymbol{\gamma}_{input}, (\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V)\mathbf{P}_1^\top, \mathbf{P}_1\mathbf{W}_O$	$\mathbb{P}_d$	Lemmas A.1.1, A.2.1	$\boldsymbol{\gamma}_{input}$	$\mathbf{C}_2\mathbf{P}_1\boldsymbol{\gamma}_{input}$	$\mathbb{P}_d \times \mathbb{R}^d_{>0}$
	$\mathbf{C}_2\boldsymbol{\gamma}_{input}, (\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V)\mathbf{C}_2^{-1}$	$\mathbb{R}^d_{>0}$	Lemma A.2.2	$\mathbf{W}_Q$	$\mathbf{P}_2\mathbf{W}_Q\mathbf{P}_1^\top\mathbf{C}_2^{-1}$	$\mathbb{P}_d \times \mathbb{R}^d_{>0}$
	$\mathbf{P}_2(\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V), \mathbf{W}_O\mathbf{P}_2^\top$	$\mathbb{P}_d$	Lemma A.2.3	$\mathbf{W}_K$	$\mathbf{P}_2\mathbf{W}_K\mathbf{P}_1^\top\mathbf{C}_2^{-1}$	$\mathbb{P}_d \times \mathbb{R}^d_{>0}$
	$\mathbf{C}_3\mathbf{W}_V, \mathbf{W}_O\mathbf{C}_3^{-1}$	$\mathbb{R}^d_{>0}$	Lemma A.2.4	$\mathbf{W}_V$	$\mathbf{C}_3\mathbf{P}_2\mathbf{W}_V\mathbf{P}_1^\top\mathbf{C}_2^{-1}$	$\mathbb{P}_d \times \mathbb{R}^d_{>0}$
				$\mathbf{W}_O$	$\mathbf{P}_1\mathbf{W}_O\mathbf{P}_2^\top\mathbf{C}_3^{-1}$	$\mathbb{P}_d \times \mathbb{R}^d_{>0}$
FFN Boundary	$\mathbf{P}_1\boldsymbol{\gamma}_{post}, (\mathbf{W}_g, \mathbf{W}_u)\mathbf{P}_1^\top, \mathbf{P}_1\mathbf{W}_d$	$\mathbb{P}_d$	Lemmas A.1.1, A.2.1	$\boldsymbol{\gamma}_{post}$	$\mathbf{C}_4\mathbf{P}_1\boldsymbol{\gamma}_{post}$	$\mathbb{P}_d \times \mathbb{R}^d_{>0}$
	$\mathbf{C}_4\boldsymbol{\gamma}_{post}, (\mathbf{W}_g, \mathbf{W}_u)\mathbf{C}_4^{-1}$	$\mathbb{R}^d_{>0}$	Lemma A.3.1	$\mathbf{W}_g$	$\mathbf{P}_3\mathbf{W}_g\mathbf{P}_1^\top\mathbf{C}_4^{-1}$	$\mathbb{P}_d \times \mathbb{R}^d_{>0}$
	$\mathbf{P}_3(\mathbf{W}_g, \mathbf{W}_u), \mathbf{W}_d\mathbf{P}_3^\top$	$\mathbb{P}_d$	Lemma A.3.2	$\mathbf{W}_u$	$\mathbf{C}_5\mathbf{P}_3\mathbf{W}_u\mathbf{P}_1^\top\mathbf{C}_4^{-1}$	$\mathbb{P}_d \times \mathbb{R}^d_{>0}$
	$\mathbf{C}_5\mathbf{W}_u, \mathbf{W}_d\mathbf{C}_5^{-1}$	$\mathbb{R}^d_{>0}$	Lemma A.3.3	$\mathbf{W}_d$	$\mathbf{P}_1\mathbf{W}_d\mathbf{P}_3^\top\mathbf{C}_5^{-1}$	$\mathbb{P}_d \times \mathbb{R}^d_{>0}$

4.1 Key insight and high-level overview

Despite their diverse embedding strategies, we observe that existing MEAs heavily rely on predictable parameter positions and stable binary representations during payload recovery. To fundamentally sanitize LLMs while preserving their functional equivalence, our key insight is to leverage symmetry transformations that randomize both parameter indices and values, thereby rendering embedded payloads irrecoverable.

Definition 4.1.1 (Symmetry transformations in LLMs)**.** Let G_i be a symmetry group that acts on each component weight matrix $\mathbf{W}_i \in \Theta$, and a group action $g_i \in G_i$ induces a transformation $g_i \cdot \mathbf{W}_i \mapsto \mathbf{W}_i'$. A collection of transformations $\{g_1, \cdots g_N\}$ constitutes *symmetry transformations* if they yield a sanitized parameter set $\Theta' = \{g_1 \cdot \mathbf{W}_1, \cdots, g_N \cdot \mathbf{W}_N\}$ that strictly preserves functional equivalence:

$$F_{\Theta'}(x) = F_\Theta(x), \quad \forall x \in \mathcal{X}. \tag{5}$$

Guided by this insight, SymSan systematically applies symmetry transformations to eliminate embedded payloads. As illustrated in Figure 1, the high-level workflow of SymSan proceeds as follows. It first identifies two fundamental symmetry groups inherent to mainstream LLM architectures, i.e., permutation ($\mathbb{P}_d$) and scaling ($\mathbb{R}^d_{>0}$). To guarantee functional equivalence, it then formulates 10 complementary transformation pairs across different model components. Within each pair, the transformations are applied jointly such that their functional effects precisely cancel out during the forward pass, strictly satisfying Eq. (5). By aggregating all pairwise transformations, SymSan comprehensively sanitizes the entire model while strictly preserving functional equivalence.

4.2 Symmetries in mainstream LLMs

In what follows, we detail two symmetries adopted by SymSan in mainstream LLM architectures.

Permutation Symmetry ($\mathbb{P}_d$) acts on weight matrices $\mathbf{W}_{in} \in \mathbb{R}^{d_{in} \times d}$ and $\mathbf{W}_{out} \in \mathbb{R}^{d \times d_{out}}$ that share a common intermediate dimension d, as well as normalization vectors $\boldsymbol{\gamma} \in \mathbb{R}^d$ associated with this dimension, defined as:

$$\mathbf{W}_{in} \mapsto \mathbf{W}_{in}\mathbf{P}^\top, \quad \mathbf{W}_{out} \mapsto \mathbf{P}\mathbf{W}_{out}, \quad \boldsymbol{\gamma} \mapsto \mathbf{P}\boldsymbol{\gamma}, \tag{6}$$

where $\mathbf{P} \in \mathbb{R}^{d \times d}$ is a permutation matrix satisfying $\mathbf{P}\mathbf{P}^\top = \mathbf{P}^\top\mathbf{P} = \mathbf{I}_d$. In essence, by randomly permuting the rows and columns of paired weight matrices, SymSan shuffles the physical storage locations of parameters while preserving their functional roles. Consequently, this symmetry fundamentally breaks the deterministic link between parameter indices and payload bits, rendering any index-dependent extraction strategy ineffective.

Scaling Symmetry ($\mathbb{R}^d_{>0}$) further perturbs the numerical values by applying random positive scaling factors to the pairs. It operates on paired weight matrices that are coupled through scale-invariant normalization or direct linear composition, defined as:

$$\begin{aligned} &\boldsymbol{\gamma} \mapsto \mathbf{C}\boldsymbol{\gamma}, \quad \mathbf{W}_{out} \mapsto \mathbf{W}_{out}\mathbf{C}^{-1}, \\ \text{or} \quad &\mathbf{W}_{in} \mapsto \mathbf{C}\mathbf{W}_{in}, \quad \mathbf{W}_{out} \mapsto \mathbf{W}_{out}\mathbf{C}^{-1} \end{aligned} \tag{7}$$

where $\mathbf{C} = \mathrm{diag}(\mathbf{c}) \in \mathbb{R}^{d \times d}$ is a diagonal matrix with entries c_i sampled uniformly from a positive range. Distinct from permutation, scaling symmetry modifies the magnitude of parameters, thereby disrupting payload bits encoded in the floating-point representation.

4.3 Performing symmetries in LLMs

To apply the aforementioned symmetry transformations to mainstream LLMs, we first identify three boundaries that collectively cover all model parameters. Based on this, we further discover 10 complementary pairs, where transformations within each pair are independent of others and strictly preserve functional equivalence. Then, SymSan instantiates permutation matrices $\mathbf{P}_i \in \mathbb{P}_d$ and scaling matrices $\mathbf{C}_i \in \mathbb{R}^d_{>0}$, controlled by random permutation seeds p_i and scaling factors $s_i = [\alpha_1, \cdots, \alpha_d]$, respectively, with each α_i randomly sampled within $(1 - r, 1 + r)$. By integrating the transformation matrices of all pairs, each parameter weight $\mathbf{W_i}$ undergoes a composite transformation and is transformed to $\mathbf{W_i}'$. In particular, the residual-stream permutation $\mathbf{P}_1$ introduced at the I-O boundary propagates through all normalization vectors and is absorbed on the input side of every intermediate projection as the right factor $\mathbf{P}_1^\top$ and on the output side as the left factor $\mathbf{P}_1$. We detail each boundary, complementary pair, and their corresponding symmetry transformations below, as summarized in Table 2.

Input-Output (I-O) Boundary: This boundary encompasses the embedding matrix $\mathbf{W}_{emb}$, the final normalization vector $\boldsymbol{\gamma}_{final}$, and LM head $\mathbf{W}_{lm}$, governed by two complementary pairs.

(1) $\mathbf{W}_{emb}$ and $\boldsymbol{\gamma}_{final}$ serve as $\mathbf{W}_{in}$, paired with the LM head $\mathbf{W}_{lm}$ as $\mathbf{W}_{out}$. They undergo a permutation transformation mapped by $\mathbf{P}_1$, resulting in $\mathbf{P}_1\mathbf{W}_{emb}$, $\mathbf{P}_1\boldsymbol{\gamma}_{final}$, and $\mathbf{W}_{lm}\mathbf{P}_1^\top$, respectively, as proven in Lemma A.1.1.

(2) $\boldsymbol{\gamma}_{final}$ and $\mathbf{W}_{lm}$ constitute a scaling pair, transformed into $\mathbf{C}_1\boldsymbol{\gamma}_{final}$ and $\mathbf{W}_{lm}\mathbf{C}_1^{-1}$ to preserve functional invariance, as verified in Lemma A.1.2.

Consequently, the composite transformations map $\mathbf{W}_{emb}$, $\boldsymbol{\gamma}_{\text{final}}$, and $\mathbf{W}_{lm}$ to $\mathbf{P}_1\mathbf{W}_{emb}$, $\mathbf{C}_1\mathbf{P}_1\boldsymbol{\gamma}_{\text{final}}$, and $\mathbf{W}_{lm}\mathbf{P}_1^\top\mathbf{C}_1^{-1}$, respectively.

Attention Boundary: This boundary includes the pre-attention normalization vector $\boldsymbol{\gamma}_{input}$ and the attention projection matrices $\mathbf{W}_Q$, $\mathbf{W}_K$, $\mathbf{W}_V$, and $\mathbf{W}_O$. Their transformations are coordinated through four complementary pairs to ensure consistency with the preceding I-O boundary and internal structural constraints.

(1) To maintain consistency with the $\mathbf{P}_1$-permuted embeddings from the I-O boundary, $\boldsymbol{\gamma}_{input}$ is transformed into $\mathbf{P}_1\boldsymbol{\gamma}_{input}$ to propagate the permutation through the normalization layer (Lemma A.2.1), while $\mathbf{W}_Q$, $\mathbf{W}_K$, and $\mathbf{W}_V$ are right-compensated by $\mathbf{P}_1^\top$ and $\mathbf{W}_O$ is left-multiplied by $\mathbf{P}_1$, so that the attention sub-block reads the permuted activations and writes its output back into the $\mathbf{P}_1$-permuted residual stream (Lemma A.1.1).

(2) A scaling symmetry is applied between $\boldsymbol{\gamma}_{input}$ and $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V$. Specifically, $\boldsymbol{\gamma}_{input}$ is scaled by $\mathbf{C}_2$, while $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V$ are compensated by $\mathbf{C}_2^{-1}$, as detailed in Lemma A.2.2.

(3) The projection weights $(\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V)$ undergo permutation transformations. $(\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V)$ are mapped via $\mathbf{P}_2$, while $\mathbf{W}_O$ is compensated by $\mathbf{P}_2^\top$. Notably, this pair is subject to the Rotary Positional Embedding (RoPE) mechanism [33], which constrains the permutation matrix $\mathbf{P}_2$ to preserve the geometric properties of relative positions, as verified in Lemma A.2.3.

(4) Finally, a scaling symmetry operates between the value projection and the output projection. $\mathbf{W}_V$ is scaled by $\mathbf{C}_3$ and $\mathbf{W}_O$ by $\mathbf{C}_3^{-1}$, preserving functional equivalence of the attention block (Lemma A.2.4).

Aggregating these transformations, $\boldsymbol{\gamma}_{input}$, $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V$, and $\mathbf{W}_O$ are respectively mapped to $\mathbf{C}_2\mathbf{P}_1\boldsymbol{\gamma}_{input}$, $\mathbf{P}_2\mathbf{W}_Q\mathbf{P}_1^\top\mathbf{C}_2^{-1}$, $\mathbf{P}_2\mathbf{W}_K\mathbf{P}_1^\top\mathbf{C}_2^{-1}$, $\mathbf{C}_3\mathbf{P}_2\mathbf{W}_V\mathbf{P}_1^\top\mathbf{C}_2^{-1}$, and $\mathbf{P}_1\mathbf{W}_O\mathbf{P}_2^\top\mathbf{C}_3^{-1}$.

FFN Boundary: This boundary covers the post-attention normalization vector $\boldsymbol{\gamma}_{post}$ and the feed-forward network weights $\mathbf{W}_g$, $\mathbf{W}_u$ and $\mathbf{W}_d$. The sanitization proceeds through four complementary pairs.

(1) Consistent with the I-O boundary, $\boldsymbol{\gamma}_{post}$ inherits the $\mathbf{P}_1$, mapping to $\mathbf{P}_1\boldsymbol{\gamma}_{post}$ to align with the incoming activations (Lemma A.2.1). Meanwhile, $\mathbf{W}_g$ and $\mathbf{W}_u$ are right-compensated by $\mathbf{P}_1^\top$ and $\mathbf{W}_d$ is left-multiplied by $\mathbf{P}_1$ (Lemma A.1.1).

(2) A scaling symmetry couples $\boldsymbol{\gamma}_{post}$ with the initial FFN projections. $\boldsymbol{\gamma}_{post}$ is scaled by $\mathbf{C}_4$, while $\mathbf{W}_g$ and $\mathbf{W}_u$ are inversely scaled by $\mathbf{C}_4^{-1}$, strictly preserving the output magnitude of the normalization layer (Lemma A.3.1).

(3) The projection weights $\mathbf{W}_g$ and $\mathbf{W}_u$ are randomized via permutation transformation using $\mathbf{P}_3$, while $\mathbf{W}_d$ is compensated by $\mathbf{P}_3^\top$. This compensatory structure ensures that the functional equivalence of the FFN layer is strictly preserved (Lemma A.3.2).

(4) Lastly, a scaling symmetry is applied between $\mathbf{W}_u$ and $\mathbf{W}_d$. $\mathbf{W}_u$ is further scaled by $\mathbf{C}_5$ and $\mathbf{W}_d$ by $\mathbf{C}_5^{-1}$, ensuring that the overall input-output mapping of the FFN block remains invariant (Lemma A.3.3).

Combining these operations, the mappings for the FFN components are derived as: $\boldsymbol{\gamma}_{post} \mapsto \mathbf{C}_4\mathbf{P}_1\boldsymbol{\gamma}_{post}$, $\mathbf{W}_g \mapsto \mathbf{P}_3\mathbf{W}_g\mathbf{P}_1^\top\mathbf{C}_4^{-1}$, $\mathbf{W}_u \mapsto \mathbf{C}_5\mathbf{P}_3\mathbf{W}_u\mathbf{P}_1^\top\mathbf{C}_4^{-1}$, and $\mathbf{W}_d \mapsto \mathbf{P}_1\mathbf{W}_d\mathbf{P}_3^\top\mathbf{C}_5^{-1}$.

It is worth noting that certain symmetries are not universally applicable across all parameter types due to their structural roles. Specifically, $\mathbf{W}_{emb}$ does not naturally admit scaling symmetry ($\mathbb{R}_{>0}^d$). However, this structural constraint does not compromise the efficacy of SymSan, as the remaining symmetries introduce sufficient parameter perturbation to mitigate real-world attacks, as empirically demonstrated in Section 5.2.3.

Furthermore, our methodology seamlessly adapts to complex architectures such as multimodal models and Mixture-of-Experts (MoE). By applying analogous complementary pairs to modality-specific embeddings and expert routers, SymSan maintains both model functional equivalence and defense effectiveness. Detailed derivations and proofs are included in Lemma A.4.1 and Lemma A.4.2.

5 Experiments and Evaluation

5.1 Experimental setting

Testbed. All experiments are conducted on a cloud server with the Ubuntu 22.04.4 LTS operating system and x86_64 architecture with 1 NVIDIA RTX A6000 Pro GPU. The software environment is PyTorch 2.3.0+cu121 framework with Python 3.10.14.

Models. We tested a multimodal LLM Gemma-3-4b-it [35], a MoE model OLMoE-1B-7B-0924-Instruct [27], as well as two dense models Mistral-7B-Instruct-v0.3 [17] and Llama-2-13b-chat-hf [37], all of which are widely adopted in real-world scenarios with diverse model sizes and architectures. We obtained their weights from Hugging Face and ran each model with default hyperparameters.

Attacks. We evaluated SymSan against a diverse spectrum of embedding attacks. Specifically, we first considered MSB [42] and LSB Substitution [8], which embed malware payloads into the most or least significant bit of the parameters within a selected layer. MalModel [16] randomly substitutes the lowest byte (8 bits) of parameters within the query weight matrices. Beyond substitutions, we considered mapping-based attacks, i.e., Value Mapping [22] and Sign Mapping [22], which encode payload bits by identifying parameters whose values or signs align with the target bits. MaleficNet [13, 14] integrates CDMA spread-spectrum and LDPC error correction under its default configuration. FREEZER [48] targets exponent bits with a 5× redundancy factor and employs dual-majority voting for fault tolerance. MEASER [34] applies its robust embedding scheme at the 6th bit position, augmented with LDPC coding and spread-spectrum modulation using default settings. Finally, we assess Resilience Training [22], a training-phase method that embeds payloads via LoRA-based fine-tuning [15] on the MMLU corpus [12] for a single epoch.

Payloads. We sourced a diverse set of prevalent real-world malware payloads from TheZoo[2], an open repository of malware families. To ensure a comprehensive evaluation, the selected binaries span a wide spectrum of malware types, target platforms, and sizes, as detailed in Table 3.

[2]https://thezoo.morirt.com/

Table 3: The malware payloads

Type	Malware	Platforms	Size
Ransomware	Ransomware.WannaCry	Windows	3.5 MB
	Linux.Encoder.1	Linux	2.3 MB
Trojan	Win32.Emotet	Windows	96.5 KB
	Linux.Mirai.B	Linux	467.1 KB
Worm	TrojanWin32.Duqu.Stuxnet	Windows	25.0 KB
	W32.MyDoom.A	Windows	736.5 KB
Rootkit	Rustock.NFE	Windows	98.2 KB
	Trojan.Regin	Windows/Linux/MacOS	173.8 KB
Spyware	Android.PegasusB	Android	1.1 MB
	Win32.ZeusVM	Windows	469.7 KB

Defense Baselines. We compared our method against a comprehensive range of sanitization defenses. We begin with randomization-based approaches: Gaussian Noise [1] corrupts embedded payloads by injecting additive Gaussian perturbations $\sigma = 0.001$ into model parameters, while CDR [6] achieves similar disruption by randomly substituting the five least significant bits of targeted weights. For compression-based defenses, Prune [13, 14] eliminates payloads by randomly zeroing out 25% of all model parameters, and Quantization (Quant.) [34] mitigates risks by reducing numerical precision via AWQ [21] algorithm. In addition, we also evaluated Fine-Tuning (FT) [48] by applying LoRA exclusively to payload-embedded weights, trained on the MMLU benchmark for 3 epochs. Finally, we included NeuPerm [9], which eliminates payloads through a permutation transformation applied specifically to the compromised parameter subsets.

Metrics. We introduce five metrics categorized into two primary aspects: defense effectiveness and model utility.

Metrics of Defense Effectiveness: We adopt the Bit Error Rate (BER) [48] as our primary evaluation metric. The BER computes the proportion of mismatched bits between the embedded and the extracted payloads, i.e., BER $= \frac{1}{l_b}\sum_{i=1}^{l_b}|b_i - b'_i|$, where l_b indicates the total bit-length of the payload. The variables b_i and b'_i correspond to the i-th bit of the ground-truth payload b and the recovered payload b', respectively. Notably, b' is obtained after the decoding procedure of each attack. A BER of 0 indicates that all bits match after decoding, implying defense failure. Conversely, a BER approaching 0.5 indicates that the decoded bits are largely uncorrelated with the original payload, suggesting that the recovered bits are indistinguishable from random guesses. While BER measures the bit mismatch of a single payload, we additionally introduce the Decode Success Rate (DSR), which measures the attack's decode success rate across multiple tested payloads. Specifically, DSR is the fraction of tested payloads that the attack successfully decodes through its own decoding procedure. A lower DSR means that more of the tested payloads fail to be decoded.

Metrics of Model Utility: To comprehensively assess the utility preservation of the sanitized LLMs, we evaluate their performance on MMLU [12], AGIEval [57], FinQA [3], and MedXpertQA[3] [60]. In particular, we incorporate three complementary metrics for quantifying model performance, i.e., accuracy (ACC), perplexity (PPL) and logit deviation (LD). To be specific, ACC and PPL measure the model's task accuracy and generation consistency, where a higher ACC indicates superior problem-solving capability, and a lower PPL signifies more confident and coherent text generation [34]. Additionally, LD measures the absolute logit difference of outputs between models before and after sanitization, where a near-zero value indicates the defense preserves near-perfect mathematical equivalence.

Experiments Setup. To ensure a fair evaluation, we fix the key hyperparameter of SymSan across all experiments. Specifically, the scaling factor range is set to $\alpha_i \in (0.5, 1.5)$ by configuring $r = 0.5$, which provides sufficient perturbation to disrupt embedded payloads while preserving model utility. SymSan applies all symmetry transformations sequentially to each parameter matrix, following the complementary pair order defined in Table 2.

5.2 Evaluation results

5.2.1 **SymSan can effectively sanitize LLMs.** As shown in Table 4, SymSan demonstrates consistent and thorough defense efficacy across all tested models and attack methods. Unlike baseline defenses that often yield a BER of 0 against robust attacks like MaleficNet or MEASER, indicating complete defense failure, SymSan consistently maintains non-zero BERs, proving its ability to disrupt embedded payloads in all evaluated scenarios. More importantly, SymSan drives the BER close to the ideal value of 0.5 across diverse models and attacks, suggesting that the recovered payload is indistinguishable from random guessing. This stability stands in stark contrast to existing methods, which suffer from significant performance instability across different architectures and attack vectors. For instance, NeuPerm's efficacy drops notably from 0.49 for MSB substitution on Gemma-3-4b-it to 0.00 for MEASER on the same model, while Noise fluctuates from 0.47 for MSB substitution on Mistral-7B-Instruct-v0.3 to merely 0.24 for the same attack on Gemma-3-4b-it. In contrast, SymSan consistently achieves at least 0.47 BER against existing MEAs, demonstrating that the embedded payload bits are thoroughly scrambled. Additionally, SymSan achieves DSR = 0 against every attack. These attacks include MaleficNet, MEASER, and FREEZER. They employ LDPC coding, spread-spectrum modulation, and majority voting, respectively, to tolerate bit flips. This confirms that none of the tested payloads can be successfully decoded, even after the error correction of these attacks.

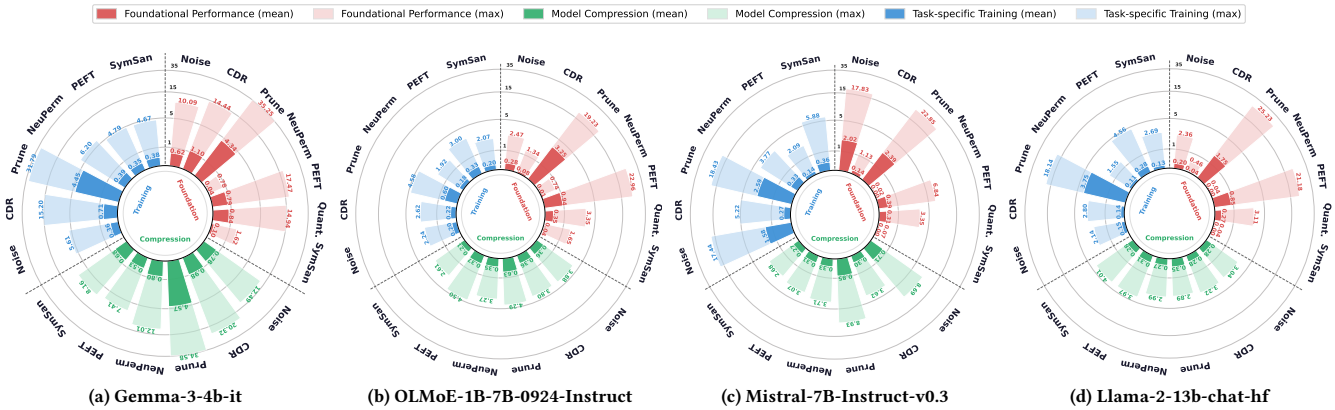
5.2.2 **SymSan can preserve equivalent model utility.** We evaluate the sanitized models' utility from two complementary perspectives: (1) the model's *foundational capability* to directly perform general tasks, and (2) the model's *downstream adaptability* to sustain performance after post-customization (i.e., model compression and task-specific training).

SymSan maintains equivalent model performance on foundational capability. As can be observed from the "Foundation" rows in Table 5, SymSan maintains ACC and PPL nearly identical to the undefended model across all four models on both datasets. Notably, on Mistral-7B-Instruct-v0.3 and Llama-2-13b-chat-hf, the ACC and PPL are exactly the same as the undefended model. This level of performance preservation is unmatched by most baseline defenses, which typically incur noticeable performance degradation. To further verify functional equivalence in practice, we evaluate the LD under the same settings. As shown in Figure 2, SymSan achieves a mean LD consistently below 0.1 across all models, and precisely 0.0

[3]We exclusively consider the text-only subset of MedXpertQA.

Table 4: The DSR and BER of SymSan and baseline defenses under various attacks across four models. Results are averaged over 3 runs with independent random seeds, with subscript indicating standard deviation. The best BER results are highlighted in bold.

Model	Gemma-3-4b-it							
Defense	None	Noise [1]	CDR [6]	Prune [13, 14]	NeuPerm [9]	FT [34, 48]	Quant. [34]	SymSan
MSB Substitution [42]	1.0/0.00$_{\pm 0.00}$	0.0/0.24$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	0.0/0.11$_{\pm 0.00}$	0.0/0.49$_{\pm 0.00}$	0.0/0.23$_{\pm 0.20}$	0.0/0.39$_{\pm 0.00}$	0.0/**0.49**$_{\pm 0.00}$
LSB Substitution [8]	1.0/0.00$_{\pm 0.00}$	0.0/0.49$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$	0.0/0.11$_{\pm 0.00}$	0.0/0.47$_{\pm 0.01}$	0.0/0.43$_{\pm 0.05}$	0.0/0.48$_{\pm 0.00}$	0.0/**0.50**$_{\pm 0.00}$
MalModel [16]	1.0/0.00$_{\pm 0.00}$	0.0/0.40$_{\pm 0.00}$	0.0/0.31$_{\pm 0.00}$	0.0/0.11$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$	0.0/0.21$_{\pm 0.03}$	0.0/0.44$_{\pm 0.00}$	0.0/**0.50**$_{\pm 0.00}$
Resilience Training [22]	1.0/0.00$_{\pm 0.00}$	0.0/0.51$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$	0.0/0.48$_{\pm 0.00}$	0.0/0.49$_{\pm 0.00}$	0.0/0.51$_{\pm 0.00}$	0.0/0.45$_{\pm 0.00}$	0.0/**0.50**$_{\pm 0.00}$
Value Mapping [22]	1.0/0.00$_{\pm 0.00}$	0.0/0.44$_{\pm 0.00}$	0.0/0.36$_{\pm 0.00}$	0.0/0.10$_{\pm 0.00}$	0.0/0.48$_{\pm 0.02}$	0.0/0.24$_{\pm 0.01}$	0.0/0.46$_{\pm 0.00}$	0.0/**0.49**$_{\pm 0.00}$
Sign Mapping [22]	1.0/0.00$_{\pm 0.00}$	0.0/0.02$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	0.0/0.47$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	0.0/**0.50**$_{\pm 0.00}$
MaleficNet [13, 14]	1.0/0.00$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	0.0/0.25$_{\pm 0.13}$	1.0/0.00$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	0.0/**0.50**$_{\pm 0.00}$
FREEZER [48]	1.0/0.00$_{\pm 0.00}$	0.0/0.09$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	0.0/0.07$_{\pm 0.00}$	0.0/0.45$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	0.0/0.25$_{\pm 0.00}$	0.0/**0.47**$_{\pm 0.00}$
MEASER [34]	1.0/0.00$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	0.0/**0.50**$_{\pm 0.00}$
Model	OLMoE-1B-7B-0924-Instruct							
MSB Substitution	1.0/0.00$_{\pm 0.00}$	0.0/0.30$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	0.0/0.11$_{\pm 0.00}$	0.0/0.48$_{\pm 0.01}$	0.0/0.10$_{\pm 0.03}$	0.0/0.39$_{\pm 0.02}$	0.0/**0.48**$_{\pm 0.01}$
LSB Substitution	1.0/0.00$_{\pm 0.00}$	0.0/0.48$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$	0.0/0.11$_{\pm 0.00}$	0.0/0.46$_{\pm 0.00}$	0.0/0.50$_{\pm 0.01}$	0.0/0.49$_{\pm 0.00}$	0.0/**0.50**$_{\pm 0.01}$
MalModel	1.0/0.00$_{\pm 0.00}$	0.0/0.44$_{\pm 0.05}$	0.0/0.31$_{\pm 0.00}$	0.0/0.11$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	0.0/**0.49**$_{\pm 0.01}$
Resilience Training	1.0/0.00$_{\pm 0.00}$	0.0/0.50$_{\pm 0.01}$	0.0/0.50$_{\pm 0.01}$	0.0/0.48$_{\pm 0.01}$	0.0/0.50$_{\pm 0.01}$	0.0/0.50$_{\pm 0.01}$	0.0/0.50$_{\pm 0.01}$	0.0/**0.50**$_{\pm 0.01}$
Value Mapping	1.0/0.00$_{\pm 0.00}$	0.0/0.48$_{\pm 0.01}$	0.0/0.30$_{\pm 0.05}$	0.0/0.10$_{\pm 0.00}$	0.0/0.42$_{\pm 0.07}$	0.0/0.47$_{\pm 0.01}$	0.0/0.46$_{\pm 0.01}$	0.0/**0.49**$_{\pm 0.00}$
Sign Mapping	1.0/0.00$_{\pm 0.00}$	0.0/0.04$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	0.0/0.49$_{\pm 0.01}$	0.0/0.01$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	0.0/**0.50**$_{\pm 0.00}$
MaleficNet	1.0/0.00$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	0.0/0.04$_{\pm 0.04}$	1.0/0.00$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	0.0/**0.50**$_{\pm 0.00}$
FREEZER	1.0/0.00$_{\pm 0.00}$	0.0/0.23$_{\pm 0.08}$	1.0/0.00$_{\pm 0.00}$	0.0/0.07$_{\pm 0.00}$	0.0/0.38$_{\pm 0.06}$	0.0/0.04$_{\pm 0.02}$	0.0/0.15$_{\pm 0.04}$	0.0/**0.48**$_{\pm 0.01}$
MEASER	1.0/0.00$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	0.0/**0.50**$_{\pm 0.00}$
Model	Mistral-7B-Instruct-v0.3							
MSB Substitution	1.0/0.00$_{\pm 0.00}$	0.0/0.47$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	0.0/0.11$_{\pm 0.00}$	0.0/0.46$_{\pm 0.02}$	0.0/0.24$_{\pm 0.01}$	0.0/0.39$_{\pm 0.00}$	0.0/**0.49**$_{\pm 0.00}$
LSB Substitution	1.0/0.00$_{\pm 0.00}$	0.0/0.47$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$	0.0/0.11$_{\pm 0.00}$	0.0/0.44$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$	0.0/0.46$_{\pm 0.00}$	0.0/**0.50**$_{\pm 0.00}$
MalModel	1.0/0.00$_{\pm 0.00}$	0.0/0.49$_{\pm 0.01}$	0.0/0.31$_{\pm 0.00}$	0.0/0.11$_{\pm 0.00}$	0.0/0.46$_{\pm 0.01}$	0.0/0.45$_{\pm 0.03}$	0.0/0.44$_{\pm 0.01}$	0.0/**0.50**$_{\pm 0.00}$
Resilience Training	1.0/0.00$_{\pm 0.00}$	0.0/0.50$_{\pm 0.01}$	0.0/0.50$_{\pm 0.01}$	0.0/0.48$_{\pm 0.01}$	0.0/0.49$_{\pm 0.01}$	0.0/0.50$_{\pm 0.01}$	0.0/0.45$_{\pm 0.00}$	0.0/**0.50**$_{\pm 0.01}$
Value Mapping	1.0/0.00$_{\pm 0.00}$	0.0/0.49$_{\pm 0.00}$	0.0/0.30$_{\pm 0.05}$	0.0/0.11$_{\pm 0.00}$	0.0/0.40$_{\pm 0.07}$	0.0/0.46$_{\pm 0.01}$	0.0/0.45$_{\pm 0.00}$	0.0/**0.50**$_{\pm 0.00}$
Sign Mapping	1.0/0.00$_{\pm 0.00}$	0.0/0.14$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	0.0/0.48$_{\pm 0.00}$	0.0/0.04$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	0.0/**0.50**$_{\pm 0.00}$
MaleficNet	1.0/0.00$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	0.0/0.38$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	0.0/**0.50**$_{\pm 0.00}$
FREEZER	1.0/0.00$_{\pm 0.00}$	0.0/0.55$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	0.0/0.07$_{\pm 0.00}$	0.0/0.52$_{\pm 0.00}$	0.0/0.03$_{\pm 0.01}$	0.0/0.60$_{\pm 0.00}$	0.0/**0.51**$_{\pm 0.01}$
MEASER	1.0/0.00$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	0.0/**0.50**$_{\pm 0.00}$
Model	Llama-2-13b-chat-hf							
MSB Substitution	1.0/0.00$_{\pm 0.00}$	0.0/0.38$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	0.0/0.11$_{\pm 0.00}$	0.0/0.47$_{\pm 0.00}$	0.0/0.13$_{\pm 0.00}$	0.0/0.42$_{\pm 0.00}$	0.0/**0.49**$_{\pm 0.00}$
LSB Substitution	1.0/0.00$_{\pm 0.00}$	0.0/0.48$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$	0.0/0.11$_{\pm 0.00}$	0.0/0.47$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$	0.0/0.45$_{\pm 0.00}$	0.0/**0.50**$_{\pm 0.00}$
MalModel	1.0/0.00$_{\pm 0.00}$	0.0/0.49$_{\pm 0.00}$	0.0/0.31$_{\pm 0.00}$	0.0/0.11$_{\pm 0.00}$	0.0/0.45$_{\pm 0.00}$	0.0/0.46$_{\pm 0.00}$	0.0/0.45$_{\pm 0.00}$	0.0/**0.50**$_{\pm 0.00}$
Resilience Training	1.0/0.00$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$	0.0/0.48$_{\pm 0.00}$	0.0/0.49$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$	0.0/0.45$_{\pm 0.00}$	0.0/**0.50**$_{\pm 0.00}$
Value Mapping	1.0/0.00$_{\pm 0.00}$	0.0/0.49$_{\pm 0.00}$	0.0/0.25$_{\pm 0.00}$	0.0/0.11$_{\pm 0.00}$	0.0/0.46$_{\pm 0.02}$	0.0/0.42$_{\pm 0.00}$	0.0/0.44$_{\pm 0.00}$	0.0/**0.50**$_{\pm 0.00}$
Sign Mapping	1.0/0.00$_{\pm 0.00}$	0.0/0.05$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	0.0/0.49$_{\pm 0.00}$	0.0/0.01$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	0.0/**0.50**$_{\pm 0.00}$
MaleficNet	1.0/0.00$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	0.0/0.45$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	0.0/**0.50**$_{\pm 0.00}$
FREEZER	1.0/0.00$_{\pm 0.00}$	0.0/0.41$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	0.0/0.07$_{\pm 0.00}$	0.0/0.54$_{\pm 0.02}$	0.0/0.02$_{\pm 0.01}$	0.0/0.61$_{\pm 0.00}$	0.0/**0.52**$_{\pm 0.01}$
MEASER	1.0/0.00$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	0.0/**0.50**$_{\pm 0.00}$



(a) Gemma-3-4b-it (b) OLMoE-1B-7B-0924-Instruct (c) Mistral-7B-Instruct-v0.3 (d) Llama-2-13b-chat-hf

Figure 2: The LD of SymSan and baseline defenses across four models, tested under foundation, model compression, and task-specific training scenarios. We recorded mean and maximum results over 3 runs with independent random seeds.

on Mistral-7B-Instruct-v0.3 and Llama-2-13b-chat-hf. However, we observe that SymSan's maximum LD is non-zero in all cases. This deviation is attributable to floating-point rounding during parameter transformation and storage, rather than any deficiency in the theoretical symmetry transformations themselves.

SymSan preserves equivalent model performance on downstream adaptability. To assess SymSan's practical impact on model utility, we further evaluate it under two representative downstream scenarios: model compression via AWQ quantization, and task-specific training over 20 epochs. As shown in the "Model compression" row of Table 5, although the ACC and PPL of all defenses degrade relative to the foundation setting due to the precision loss introduced by quantization, SymSan consistently yields performance closest to the no-defense baseline across all evaluated models, confirming

Table 5: The ACC and PPL of models after SymSan and baseline defenses across diverse evaluation tasks. The results of task-specific training are the best performance obtained after 20 fine-tuning epochs.

Model		Gemma-3-4b-it							
Defense		None	Noise	CDR	Prune	NeuPerm	FT	Quant.	SymSan
Foundation performance	MMLU	0.60/16.32	0.58/15.15	0.55/16.45	0.28/1.06e7	0.60/16.31	0.60/9.75	0.49/18.28	0.59/16.34
	AGIEval	0.42/12.76	0.32/12.05	0.32/12.82	0.28/2.31e7	0.40/12.75	0.38/8.54	0.28/13.58	0.42/12.83
Model compression	MMLU	0.58/17.24	0.49/18.56	0.42/26.34	0.27/8.02e7	0.52/17.85	0.53/10.60	-/-	0.58/16.36
	AGIEval	0.36/13.43	0.28/13.67	0.28/20.10	0.26/1.18e8	0.30/13.22	0.31/9.03	-/-	0.35/12.76
Task-specific training	FinQA	0.21/10.47	0.22/11.08	0.23/11.75	0.00/1785.27	0.21/10.40	0.21/10.58	-/-	0.21/10.54
	MedXpertQA	0.27/22.42	0.24/23.49	0.29/22.22	0.11/1.99e11	0.22/21.28	0.29/21.01	-/-	0.26/20.02
Model		OLMoE-1B-7B-0924-Instruct							
Foundation performance	MMLU	0.50/9.88	0.49/9.84	0.48/9.92	0.23/1.13e5	0.50/9.88	0.53/10.07	0.47/10.31	0.49/9.89
	AGIEval	0.23/9.96	0.32/9.96	0.25/10.04	0.25/1.35e5	0.23/9.96	0.29/12.68	0.34/10.23	0.25/9.94
Model compression	MMLU	0.49/10.00	0.48/10.51	0.48/10.49	0.21/1.60e5	0.45/10.32	0.51/10.47	-/-	0.49/10.02
	AGIEval	0.28/10.52	0.31/10.51	0.37/10.38	0.29/1.64e5	0.32/10.20	0.31/12.75	-/-	0.34/9.98
Task-specific training	FinQA	0.08/26.84	0.07/27.52	0.06/28.09	0.00/1.56e5	0.08/26.75	0.13/24.50	-/-	0.13/26.69
	MedXpertQA	0.13/14.02	0.13/8.62	0.13/10.74	0.09/6.24e5	0.14/9.44	0.10/11.63	-/-	0.13/9.48
Model		Mistral-7B-Instruct-v0.3							
Foundation performance	MMLU	0.61/6.84	0.25/1188.30	0.61/6.88	0.26/3.59e4	0.61/6.84	0.59/6.49	0.59/7.18	0.61/6.84
	AGIEval	0.31/7.05	0.26/825.81	0.26/7.08	0.22/3.38e4	0.31/7.05	0.25/7.43	0.26/7.26	0.31/7.05
Model compression	MMLU	0.54/7.17	0.24/1740.00	0.57/7.23	0.29/3.41e4	0.60/7.18	0.59/6.70	-/-	0.58/7.17
	AGIEval	0.27/7.34	0.28/1329.83	0.35/7.31	0.26/3.14e4	0.31/7.21	0.23/7.65	-/-	0.29/7.37
Task-specific training	FinQA	0.23/7.62	0.12/16.18	0.26/8.24	0.02/2359.78	0.22/7.96	0.18/7.74	-/-	0.23/7.86
	MedXpertQA	0.24/5.42	0.09/12341.08	0.24/5.24	0.10/2.57e7	0.21/6.58	0.22/5.01	-/-	0.22/7.14
Model		Llama-2-13b-chat-hf							
Foundation performance	MMLU	0.51/8.45	0.51/8.49	0.51/8.48	0.25/9221.16	0.51/8.45	0.49/9.41	0.48/8.56	0.51/8.45
	AGIEval	0.26/8.57	0.28/8.62	0.26/8.58	0.26/8951.67	0.26/8.57	0.32/11.30	0.31/8.53	0.26/8.57
Model compression	MMLU	0.50/8.64	0.49/8.60	0.48/8.61	0.25/7402.12	0.49/8.58	0.45/9.67	-/-	0.49/8.69
	AGIEval	0.28/9.02	0.28/8.55	0.28/8.53	0.20/7428.74	0.25/8.50	0.37/11.37	-/-	0.31/8.59
Task-specific training	FinQA	0.23/6.89	0.22/7.09	0.23/6.89	0.00/2617.27	0.20/6.71	0.22/6.29	-/-	0.23/6.81
	MedXpertQA	0.20/5.38	0.18/5.56	0.20/5.40	0.07/1.92e7	0.18/5.26	0.22/4.45	-/-	0.21/5.48

that SymSan's transformations do not introduce any additional compression-induced degradation. Regarding LD in Figure 2, SymSan's values increase modestly under compression compared to the foundation setting, which is expected: quantization inherently breaks exact floating-point equivalence at the parameter level. Nevertheless, SymSan's LD remains among the lowest across all conditions, demonstrating its robustness in preserving model utility under model compression scenario. Similarly, task-specific training yields consistent results. As shown in the "Task-specific training" rows of Table 5, where models are fine-tuned via LoRA on two domain-specific datasets, SymSan-protected models achieve ACC and PPL on par with the undefended baseline across all four models. This result demonstrates that SymSan imposes no overhead on convergence speed or final task performance. However, the LD in Figure 2 also exhibits a slight increase, which is attributed to the breakdown of mathematical equivalence caused by parameter updates during fine-tuning. Taken together, these results confirm that SymSan imposes no penalty on downstream adaptability, whether via model compression or task-specific training on specialized corpora.

5.2.3 **Effectiveness under adaptive attacks.** We also consider the scenario where the adversary knows the details of SymSan and designs adaptive attacks. Specifically, in this scenario, we assume that the adversary knows the symmetry group $G = \mathbb{P}_d \times \mathbb{R}^d_{>0}$ and how each symmetry transformation is applied to each matrix, but does not know the random seeds p_i and the scaling factors s_i.

Under this scenario, we design two concrete adaptive attacks that exploit the invariant subspaces of SymSan's symmetry transformations.

- *Permutation-adaptive attacks (PAA)* target the embedding layer of LLMs, where SymSan only applies row permutation to the embedding matrix $\mathbf{W}_{\text{emb}} \in \mathbb{R}^{d\times V}$. The adversary exploits the fact that row permutation only shuffles the order of entire rows, while keeping the entries within each row unchanged. Consequently, any attack capable of recovering the original row order can fully reconstruct the embedded payload. Accordingly, we adopt a sorted-LSB variant of the standard LSB attack. For each row, the adversary reserves the first k entries as a stable per-row fingerprint, sorts all rows by this fingerprint to obtain a canonical order, and writes n payload bits into the n least-significant bits of each of the remaining $(V - k)$ entries in this canonical order. After SymSan permutes the rows, the leading entries of each row are unchanged, the canonical row order is recovered by re-sorting, and the payload is read off bit-perfectly.
- *Joint-Adaptive Attacks (JAA)* target parameters combined with both permutation and scaling transformations. The adversary exploits an overlooked algebraic invariant: if all entries in a column are set to share the same sign, this column-uniform sign survives any row permutation and positive scaling, since reordering and rescaling never change an entry's sign. For each of the N permutation units (column groups), the adversary sets the signs of all entries in the first $\lceil \log_2 N \rceil$ columns to encode the unit's original index in binary, and writes one payload bit into the column-uniform sign of each remaining column. After the defense by SymSan, the

adversary reads the fingerprints to recover the original unit order, and reassembles the payload from the column-uniform sign, one bit per column.

To assess SymSan's effectiveness under PAA and JAA, we instantiated both attacks against SymSan on four models. Specifically, PAA targets $\mathbf{W}_{emb}$ with $k = 4$ and $n = 1$, while JAA targets $\mathbf{W}_O$ with N being the number of column groups permuted by $\mathbf{P}_2$, where the constraint of contiguous column groups is derived from the RoPE restriction. We further extended both attacks with payloads encoded by error correction codes (ECC) using the encoding scheme of MEASER, denoted as PAA_{ECC} and JAA_{ECC}. The results are reported in Table 6, where both PAA and JAA can bypass SymSan when embedding locations and payload sizes are unrestricted, achieving a DSR of 1 and BER of 0 across all models. However, in practice, both PAA and JAA face severe limitations against SymSan, primarily due to the drastic compression of their available payload hiding space. For PAA, SymSan drastically reduces the invariant parameter space from the full model to only $\mathbf{W}_{emb}$, compressing the available payload hiding space from GB to MB size. For example, under our experimental settings, PAA can embed at most 19.5 MB of payload within the invariant subspace of $\mathbf{W}_{emb}$ in the 24.2 GB Llama-2-13b-chat-hf model. Notably, such a drastic shrinkage of the feasible embedding space greatly narrows the attack surface, enables targeted and cost-efficient defense, and further downgrades the overall success rate of adaptive attacks. The payload compression becomes even more severe for JAA, which can only embed a payload of up to 23.8 KB within the Llama-2-13b-chat-hf model, making it insufficient to accommodate even the smallest malware such as TrojanWin32.Duqu.Stuxnet. The underlying reason is that per-column sign is the only invariant of JAA under SymSan's joint permutation-and-scaling transform, which compresses each weight column into a single recoverable bit and restricts the total payload capacity to the KB regime. Additionally, introducing ECC does not alter the DSR and BER, as both PAA_{ECC} and JAA_{ECC} still achieve a DSR of 1 and BER of 0 in Table 6. However, the ECC encoding further compresses the effective payload capacity. In our settings, the pre-encoding payload capacity drops to 1.6 MB for PAA_{ECC} and merely 1.6 KB for JAA_{ECC}, since redundant bits consume much of the already limited embedding space, leaving even less space for real malware and further restricting practical attacks.

Overall, despite the strong threat model assumed for these adaptive attacks, SymSan effectively constrains the viable payload embedding space to merely megabytes in the best case and to merely tens of kilobytes in the worst case, with the latter rendering even the smallest real-world malware infeasible to embed in practice.

To further defeat the adaptive attacks, we introduce a lightweight extension of SymSan, termed SymSan$^+$.

- *SymSan*$^+$ extends SymSan with two additional symmetry transformations. First, it introduces a per-token scaling transformation $\mathbf{W}_{emb} \mapsto \mathbf{P}_1\mathbf{W}_{emb}\mathbf{C}_e$ where $\mathbf{C}_e = \text{diag}(c_1, \ldots, c_V)$ with each c_i randomly sampled within $(1 - r, 1 + r)$. At inference, this scaling is exactly compensated by multiplying the embedding output of token i by $1/c_i$, where $1/c_i$ is the token-specific scalar from a length-V vector stored alongside the sanitized weights. Thus, SymSan$^+$ remains exactly functionally equivalent to SymSan. Second, SymSan$^+$ introduces sign-flip symmetry [52] to every matrix subject to scaling symmetry in SymSan, applied in the same manner. Specifically, each scaling factor is drawn from $[-(1+r), -(1-r)] \cup [1-r, 1+r]$ rather than from $(1-r, 1+r)$ alone. This makes the sign bits within each scaling unit variable, while the mathematical equivalence is still preserved.

As shown in Table 6, SymSan$^+$ achieves DSR of 0 and BER of approximately 0.5 against both PAA and JAA across all models, confirming that the adaptive attacks are effectively defeated as SymSan$^+$ theoretically eliminates the invariant subspaces on which PAA and JAA critically depend.

5.3 Ablation studies

5.3.1 **Ablation of symmetry groups.** We compare SymSan against two variants, where PermSan uses only the permutation symmetry of SymSan, while ScalSan uses only its scaling symmetry. As shown in Table 6, PermSan achieves defense effectiveness comparable to SymSan across existing attacks, whereas ScalSan fails against Sign Mapping and MaleficNet. Yet this observation does not mean scaling symmetry is unimportant, as PermSan still retains a limited yet practically sufficient invariant parameter subspace for embedding real-world malware payloads under PAA (detailed in Section 5.2.3). By contrast, the joint permutation-and-scaling transformation mitigates this by drastically reducing the vulnerable invariant subspace.

5.3.2 **Ablation of scaling factors.** We further investigate the hyperparameter r that controls the sampling range of scaling factors $\alpha_i \in (1 - r, 1 + r)$. We begin by evaluating its impact on defense effectiveness. Since permutation symmetry alone already provides sufficient defense against existing attacks, we specifically examine r's impact on the scaling-only variant (ScalSan). As shown in Figure 3a, when $r \geq 0.5$, ScalSan achieves BER close to 0.5 across all attacks except sign mapping and those leveraging robust encoding, where defense failure is intrinsic to scaling symmetry itself rather than dependent on r. This indicates that $r \geq 0.5$ is sufficient to effectively disrupt parameter values through scaling transformations. Moreover, we evaluate the impact of r on model utility during task-specific training. As illustrated in Figures 3b, 3c, and 3d, when $0 < r < 1$, the ACC, PPL, and LD of all four models remain bounded and non-degenerate, except for Gemma-3-4b-it, which degrades as r approaches 1. However, when $r = 1$, corresponding to scaling factors in the range $(0, 2)$, both PPL and LD degrade to NaN, and ACC drops to near-zero. This degradation stems from excessive scaling factors that cause numerical explosion during forward propagation, particularly when combined with gradient updates during fine-tuning. Additionally, we observe that when $r > 0.5$, Gemma-3-4b-it exhibits gradually declining ACC, suggesting that its architecture is more sensitive to scaling perturbations compared to the other models.

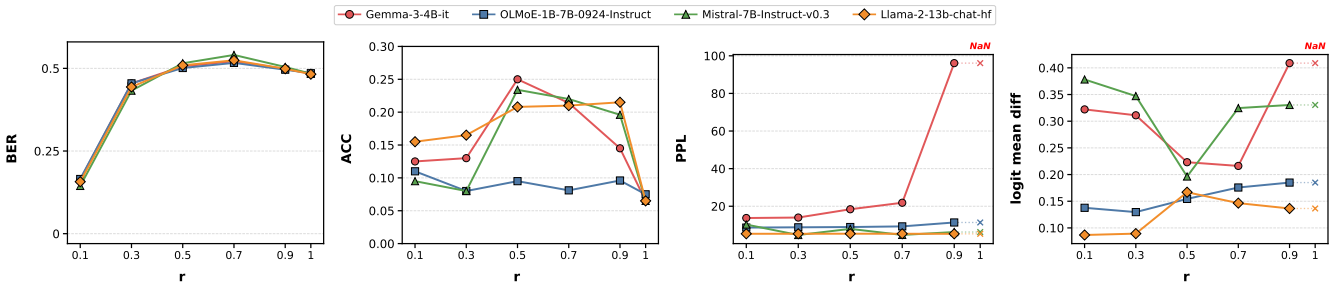
5.4 Computational cost

We evaluate the computational overhead of SymSan across four models, as presented in Figure 4. The results show that SymSan achieves a substantially lower runtime than PEFT and quantization baselines, which require extra data training and validation. While SymSan is slightly slower than data-independent methods such

Table 6: The DSR and BER of SymSan and its ablation variants, and SymSan$^+$ (extension) under existing and adaptive attacks. Results are averaged over 3 runs with independent random seeds.

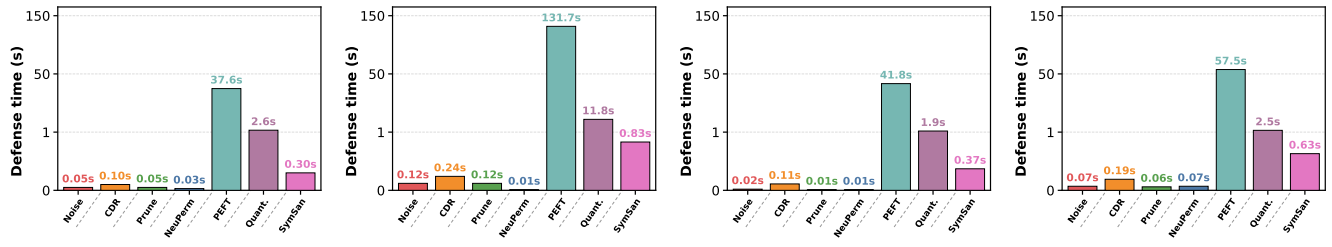
Model	Gemma-3-4b-it				OLMoE-1B-7B-0924-Instruct			
Defense	PermSan	ScalSan	SymSan	SymSan$^+$	PermSan	ScalSan	SymSan	SymSan$^+$
MSB Sub.	0.0/0.50$_{\pm 0.00}$	0.0/0.53$_{\pm 0.00}$	0.0/0.49$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$	0.0/0.54$_{\pm 0.00}$	0.0/0.48$_{\pm 0.01}$	0.0/0.50$_{\pm 0.00}$
LSB Sub.	0.0/0.50$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$	0.0/0.49$_{\pm 0.00}$	0.0/0.50$_{\pm 0.01}$	0.0/0.50$_{\pm 0.00}$
MalModel	0.0/0.50$_{\pm 0.00}$	0.0/0.48$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$	0.0/0.48$_{\pm 0.00}$	0.0/0.49$_{\pm 0.01}$	0.0/0.50$_{\pm 0.00}$
Resi. Tr.	0.0/0.50$_{\pm 0.00}$	0.0/0.28$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$	0.0/0.28$_{\pm 0.00}$	0.0/0.50$_{\pm 0.01}$	0.0/0.50$_{\pm 0.00}$
Value Map.	0.0/0.50$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$	0.0/0.49$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$	0.0/0.49$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$
Sign Map.	0.0/0.50$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$
MaleficNet	0.0/0.50$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$
FREEZER	0.0/0.50$_{\pm 0.00}$	0.0/0.08$_{\pm 0.00}$	0.0/0.47$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$	0.0/0.06$_{\pm 0.00}$	0.0/0.48$_{\pm 0.01}$	0.0/0.50$_{\pm 0.00}$
MEASER	0.0/0.50$_{\pm 0.00}$	0.0/0.16$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$	0.0/0.49$_{\pm 0.00}$	0.0/0.51$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$
PAA*	1.0/0.00$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$
PAA$_{ECC}$	1.0/0.00$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$
JAA†	1.0/0.00$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	0.0/0.50$_{\pm 0.01}$	1.0/0.00$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	0.0/0.50$_{\pm 0.02}$
JAA$_{ECC}$	1.0/0.00$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	0.0/0.50$_{\pm 0.02}$	1.0/0.00$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	0.0/0.51$_{\pm 0.02}$
Model	Mistral-7B-Instruct-v0.3				Llama-2-13b-chat-hf			
MSB Sub.	0.0/0.50$_{\pm 0.00}$	0.0/0.53$_{\pm 0.00}$	0.0/0.49$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$	0.0/0.53$_{\pm 0.00}$	0.0/0.49$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$
LSB Sub.	0.0/0.50$_{\pm 0.00}$	0.0/0.49$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$
MalModel	0.0/0.50$_{\pm 0.00}$	0.0/0.48$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$
Resi. Tr.	0.0/0.50$_{\pm 0.00}$	0.0/0.28$_{\pm 0.00}$	0.0/0.50$_{\pm 0.01}$	0.0/0.50$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$	0.0/0.38$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$
Value Map.	0.0/0.50$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$	0.0/0.46$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$
Sign Map.	0.0/0.50$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$
MaleficNet	0.0/0.50$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$
FREEZER	0.0/0.50$_{\pm 0.00}$	0.0/0.05$_{\pm 0.00}$	0.0/0.51$_{\pm 0.01}$	0.0/0.50$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$	0.0/0.06$_{\pm 0.00}$	0.0/0.52$_{\pm 0.01}$	0.0/0.50$_{\pm 0.00}$
MEASER	0.0/0.50$_{\pm 0.00}$	0.0/0.57$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$	0.0/0.50$_{\pm 0.01}$	0.0/0.50$_{\pm 0.01}$	0.0/0.49$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$
PAA*	1.0/0.00$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$
PAA$_{ECC}$	1.0/0.00$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	0.0/0.50$_{\pm 0.00}$
JAA†	1.0/0.00$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	0.0/0.51$_{\pm 0.01}$	1.0/0.00$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	0.0/0.50$_{\pm 0.01}$
JAA$_{ECC}$	1.0/0.00$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	0.0/0.48$_{\pm 0.01}$	1.0/0.00$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	1.0/0.00$_{\pm 0.00}$	0.0/0.49$_{\pm 0.02}$

* The payload embedding is restricted to the embedding layer of the LLM, with maximum payload sizes of 19.5 MB for PAA and 1.6 MB for PAA$_{ECC}$.
† The maximum payload sizes are limited to 23.8 KB for JAA and 1.6 KB for JAA$_{ECC}$, both of which are insufficient to embed our smallest malware sample (25.0 KB).



(a) BER (b) ACC (c) PPL (d) LD

Figure 3: Defense effectiveness and model utility with different r values. BER is the worst-case value across MSB/LSB Substitution, MalModel, Resilience Training, and Value Mapping attacks. ACC, PPL, and LD are averaged over the last 5 epochs of 20-epoch task-specific training on MedXpertQA.



(a) Gemma-3-4b-it (b) OLMoE-1B-7B-0924-Instruct (c) Mistral-7B-Instruct-v0.3 (d) Llama-2-13b-chat-hf

Figure 4: Computational costs of SymSan and baseline defenses across four models.

as Noise, its latency remains well below 1 second for all tested models with a maximum value of 0.83s, confirming its practicality. Moreover, the computational cost of SymSan scales with model size and architectural complexity. For dense models, SymSan takes 0.63s on Llama-2-13b-chat-hf and 0.37s on Mistral-7B-Instruct-v0.3. This discrepancy indicates a linear correlation with the overall parameter count. For the MoE model OLMoE-1B-7B-0924-Instruct, SymSan records a runtime of 0.83s, which is higher than that of

Mistral-7B-Instruct-v0.3 despite their comparable total parameter counts. The reason for this extra overhead lies in the additional FFN layers unique to MoE architectures. Overall, SymSan relies solely on matrix multiplication, yielding polynomial-time complexity that is adaptable to various model sizes and architectures, as further validated by our empirical efficiency measurements.

6 Discussion

Discussion on adaptability in conventional models. To the best of our knowledge, SymSan represents the first effort to systematically apply parameter space symmetry across all modules of modern LLMs. Prior research on parameter space symmetry has predominantly focused on conventional DNNs, e.g., feedforward networks [11] and ReLU networks [31], where various complementary pairs and symmetry transformations have been identified. Accordingly, extending SymSan to conventional models is straightforward. Specifically, SymSan's complementary pairs framework can be adapted by identifying the symmetry groups inherent to the target architecture and applying pairwise transformations that preserve functional equivalence. However, a critical prerequisite for successful sanitization is that the target model exhibits parameter space symmetry, since certain model designs intentionally remove these symmetries [59], thereby limiting the applicability of symmetry-based defenses.

Limitations and future work. Although SymSan builds a principled framework for LLM sanitization by exploiting parameter-space symmetry, it still exhibits several limitations, leaving ample room for future in-depth exploration and extension. SymSan currently relies solely on permutation and scaling symmetries. Expanding to richer symmetries like rotation [49] or sign-flip [52] could further shrink the invariant subspace that adaptive attacks exploit, though structural constraints on components may prevent their direct adoption in LLMs. SymSan can be further extended by integrating it with orthogonal defenses for enhanced overall robustness. Incorporating existing defenses, such as Noise, to specifically defending against the invariant subspaces of SymSan, can further improve the sanitization effect while keeping the model usability almost unchanged.

7 Conclusion

In this paper, we proposed SymSan, an effective LLM sanitization method grounded in parameter space symmetry theory, to defend against malware embedding attacks (MEAs). Specifically, SymSan decomposes mainstream LLM architectures into a principled set of complementary component pairs, and applies rigorously designed pairwise permutation and scaling symmetry transformations within each pair. By aggregating all pairwise transformations, each parameter matrix undergoes a composite transformation, thoroughly shuffling both indices and values to eliminate embedded payloads, while strictly preserving model functional equivalence. Extensive evaluations across diverse LLMs against a wide spectrum of MEAs demonstrate that SymSan achieves at least 0.47 BER against existing MEAs, while maintaining equivalent performance on both foundational and downstream tasks, significantly outperforming all baselines. Our findings demonstrate that parameter space symmetry provides a principled and theoretically grounded approach to LLM sanitization, offering a promising direction for both security and utility preservation in open-source model deployments.

References

[1] Guy Amit, Moshe Levy, and Yisroel Mirsky. 2024. Transpose Attack: Stealing Datasets with Bidirectional Training. In *31st Annual Network and Distributed System Security Symposium*. Internet Society.

[2] Yihong Chen, Xiangxiang Xu, Pontus Stenetorp, Sebastian Riedel, and Luca Franceschi. 2026. Decomposing LLM Computation with Jets. In *Proceedings of the 14th International Conference on Learning Representations*.

[3] Zhiyu Chen, Wenhu Chen, Charese Smiley, Sameena Shah, Iana Borova, Dylan Langdon, Reema Moussa, Matt Beane, Ting-Hao (Kenneth) Huang, Bryan Routledge, and William Yang Wang. 2021. FinQA: A Dataset of Numerical Reasoning over Financial Data. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 3697–3711.

[4] Gabriele Digregorio, Marco Di Gennaro, Stefano Zanero, Stefano Longari, and Michele Carminati. 2026. On the (In)Security of Loading Machine Learning Models. In *2026 IEEE Symposium on Security and Privacy (SP)*. IEEE, 272–289.

[5] Yupei Du and Dong Nguyen. 2023. Measuring the Instability of Fine-Tuning. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 6209–6230.

[6] Ran Dubin. 2023. Disarming Attacks Inside Neural Network Models. *IEEE Access* 11 (2023), 124295–124303.

[7] Chongzhou Fang, Ning Miao, Shaurya Srivastav, Jialin Liu, Ruoyu Zhang, Ruijie Fang, Asmita, Ryan Tsang, Najmeh Nazari, Han Wang, and Houman Homayoun. 2024. Large Language Models for Code Analysis: Do LLMs Really Do Their Job?. In *Proceedings of the 33rd USENIX Security Symposium*. USENIX Association, 829–846.

[8] Daniel Gilkarov and Ran Dubin. 2024. Steganalysis of AI Models LSB Attacks. *IEEE Transactions on Information Forensics and Security* 19 (2024), 4767–4779. doi:10.1109/TIFS.2024.3383770

[9] Daniel Gilkarov and Ran Dubin. 2025. NeuPerm: Disrupting Malware Hidden in Neural Network Parameters by Leveraging Permutation Symmetry. arXiv:2510.20367 [cs.CR]

[10] Daniel Gilkarov and Ran Dubin. 2026. Model X-Ray: Detection of hidden malware in AI model weights using few shot learning. *Journal of Information Security and Applications* 101, Article 104517 (2026).

[11] Robert Hecht-Nielsen. 1990. On the algebraic structure of feedforward network weight spaces. In *Advanced Neural Computers*. North-Holland, 129–135.

[12] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2021. Measuring Massive Multitask Language Understanding. In *Proceedings of the 9th International Conference on Learning Representations*.

[13] Dorjan Hitaj, Giulio Pagnotta, Fabio De Gaspari, Sediola Ruko, Briland Hitaj, Luigi V. Mancini, and Fernando Perez-Cruz. 2025. Do You Trust Your Model? Emerging Malware Threats in the Deep Learning Ecosystem. *IEEE Transactions on Dependable and Secure Computing* 22, 6 (2025), 6504–6521. doi:10.1109/TDSC.2025.3586703

[14] Dorjan Hitaj, Giulio Pagnotta, Briland Hitaj, Luigi V. Mancini, and Fernando Pérez-Cruz. 2022. MaleficNet: Hiding Malware into Deep Neural Networks Using Spread-Spectrum Channel Coding. In *27th European Symposium on Research in Computer Security (ESORICS 2022)*, Vol. 13556. Springer Nature Switzerland, 425–444.

[15] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *Proceedings of the 10th International Conference on Learning Representations*.

[16] Jiayi Hua, Kailong Wang, Meizhen Wang, Guangdong Bai, Xiapu Luo, and Haoyu Wang. 2026. MalModel: hiding malicious payload in mobile deep learning models with black-box backdoor attack. *Automated Software Engineering* 33, 1, Article 28 (May 2026). doi:10.1007/s10515-025-00569-7

[17] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, et al. 2023. Mistral 7B. arXiv:2310.06825 [cs.CL]

[18] Jabari Kwesi, Jiaxun Cao, Riya Manchanda, and Pardis Emami-Naeini. 2025. Exploring User Security and Privacy Attitudes and Concerns Toward the Use of General-Purpose LLM Chatbots for Mental Health. In *Proceedings of the 34th USENIX Security Symposium*. USENIX Association, 6007–6024.

[19] Derek Lim, Theo (Moe) Putterman, Robin Walters, Haggai Maron, and Stefanie Jegelka. 2024. The Empirical Impact of Neural Parameter Symmetries, or Lack Thereof. In *Proceedings of the 38th International Conference on Neural Information Processing Systems*. Curran Associates, Inc.

[20] Jie Lin and David Mohaisen. 2025. From Large to Mammoth: A Comparative Evaluation of Large Language Models in Vulnerability Detection. In *32nd Annual Network and Distributed System Security Symposium*. Internet Society.

[21] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. 2024. AWQ: Activation-aware Weight Quantization for On-Device LLM Compression and Acceleration. In *Proceedings of Machine Learning and Systems 6 (MLSys 2024)*. 87–100.

[22] Tao Liu, Zihao Liu, Qi Liu, Wujie Wen, Wenyao Xu, and Ming Li. 2020. StegoNet: Turn Deep Neural Network into a Stegomalware. In *Proceedings of the 36th Annual Computer Security Applications Conference*. Association for Computing Machinery, 928–938.

[23] Tong Liu, Guozhu Meng, Peng Zhou, Zizhuang Deng, Shuaiyin Yao, and Kai Chen. 2026. The Art of Hide and Seek: Making Pickle-Based Model Supply Chain Poisoning Stealthy Again. In *Proceedings of the 35th USENIX Security Symposium*. USENIX Association, 161–180.

[24] Razvan Marinescu, Victoria-Elisabeth Gruber, and Diego Fajardo Vargas. 2026. Medical Interpretability and Knowledge Maps of Large Language Models. In *Proceedings of the 14th International Conference on Learning Representations*.

[25] Javier Maass Martínez and Joaquín Fontbona. 2024. Symmetries in Overparametrized Neural Networks: A Mean Field View. In *Proceedings of the 38th International Conference on Neural Information Processing Systems*. Curran Associates, Inc.

[26] Fengran Mo, Chuan Meng, Mohammad Aliannejadi, and Jian-Yun Nie. 2025. Conversational Search: From Fundamentals to Frontiers in the LLM Era. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '25)*. Association for Computing Machinery, 4094–4097.

[27] Niklas Muennighoff, Luca Soldaini, Dirk Groeneveld, Kyle Lo, Jacob Morrison, Sewon Min, Weijia Shi, Evan Pete Walsh, Oyvind Tafjord, Nathan Lambert, et al. 2025. OLMoE: Open Mixture-of-Experts Language Models. In *Proceedings of the 13th International Conference on Learning Representations*.

[28] Khoa Nguyen, Khiem Ton, NhatHai Phan, Issa Khalil, Khang Tran, Cristian Borcea, Ruoming Jin, Abdallah Khreishah, and My T. Thai. 2026. NOIR: Privacy-Preserving Generation of Code with Open-Source LLMs. In *Proceedings of the 35th USENIX Security Symposium*. USENIX Association.

[29] Branislav Pecher, Jan Cegin, Robert Belanec, Jakub Simko, Ivan Srba, and Maria Bielikova. 2024. Fighting Randomness with Randomness: Mitigating Optimisation Instability of Fine-Tuning using Delayed Ensemble and Noisy Interpolation. In *Findings of the Association for Computational Linguistics: EMNLP 2024*. Association for Computational Linguistics, 11005–11044.

[30] Jie Peng, Hongwei Yang, Jing Zhao, Hengji Dong, Hui He, Weizhe Zhang, and Haoyu He. 2025. Circumventing Backdoor Space via Weight Symmetry. In *Proceedings of the 42nd International Conference on Machine Learning*. PMLR, 48886–48922.

[31] Mary Phuong and Christoph H. Lampert. 2020. Functional vs. parametric equivalence of ReLU networks. In *Proceedings of the 8th International Conference on Learning Representations*.

[32] Guohao Shen. 2024. Exploring the Complexity of Deep Neural Networks through Functional Equivalence. In *Proceedings of the 41st International Conference on Machine Learning*. PMLR, 44643–44664.

[33] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. 2024. RoFormer: Enhanced transformer with Rotary Position Embedding. *Neurocomputing* 568, Article 127063 (2024).

[34] Ming Tan, Wei Li, Hu Tao, Hailong Ma, Aodi Liu, Qian Chen, and Zilong Wang. 2026. MEASER: Malware embedding attacks on open-source LLMs. arXiv:2510.10486 [cs.CR]

[35] Gemma Team, Aishwarya Kamath, Johan Ferret, Shreya Pathak, Nino Vieillard, Ramona Merhej, Sarah Perrin, Tatiana Matejovicova, Alexandre Ramé, Morgane Rivière, et al. 2025. Gemma 3 Technical Report. arXiv:2503.19786 [cs.CL]

[36] Alexander Theus, Alessandro Cabodi, Sotiris Anagnostidis, Antonio Orvieto, Sidak Pal Singh, and Valentina Boeva. 2025. Generalized Linear Mode Connectivity for Transformers. In *Proceedings of the 39th International Conference on Neural Information Processing Systems*. Curran Associates, Inc.

[37] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. 2023. Llama 2: Open Foundation and Fine-Tuned Chat Models. arXiv:2307.09288 [cs.CL]

[38] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is All you Need. In *Proceedings of the 31st International Conference on Neural Information Processing Systems*. Curran Associates, Inc., 5998–6008.

[39] Bolun Wang, Yuanshun Yao, Shawn Shan, Huiying Li, Bimal Viswanath, Haitao Zheng, and Ben Y. Zhao. 2019. Neural Cleanse: Identifying and Mitigating Backdoor Attacks in Neural Networks. In *2019 IEEE Symposium on Security and Privacy (SP)*. IEEE, 707–723.

[40] Longwei Wang, Ifrat Ikhtear Uddin, KC Santosh, Chaowei Zhang, Xiao Qin, and Yang Zhou. 2025. Bridging Symmetry and Robustness: On the Role of Equivariance in Enhancing Adversarial Robustness. In *Proceedings of the 39th International Conference on Neural Information Processing Systems*. Curran Associates, Inc.

[41] Zhi Wang, Chaoge Liu, and Xiang Cui. 2021. EvilModel: Hiding Malware Inside of Neural Network Models. In *2021 IEEE Symposium on Computers and Communications (ISCC)*. IEEE, 1–7.

[42] Zhi Wang, Chaoge Liu, Xiang Cui, Jie Yin, and Xutong Wang. 2022. EvilModel 2.0: Bringing Neural Network Models into Malware Attacks. *Computers & Security* 120 (2022), 102807.

[43] Jiangxing Wu. 2022. Cyberspace Endogenous Safety and Security. *Engineering* 15, 8 (2022), 179–185.

[44] Jiangxing Wu. 2022. Development paradigms of cyberspace endogenous safety and security. *Science China Information Sciences* 65, 5 (2022), 156301.

[45] Yu Xia, Subhojyoti Mukherjee, Zhouhang Xie, Junda Wu, Xintong Li, Ryan Aponte, Hanjia Lyu, Joe Barrow, Hongjie Chen, Franck Dernoncourt, et al. 2025. From Selection to Generation: A Survey of LLM-based Active Learning. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 14552–14569.

[46] YuQing Xie and Tess Smidt. 2025. A Tale of Two Symmetries: Exploring the Loss Landscape of Equivariant Models. In *Proceedings of the 39th International Conference on Neural Information Processing Systems*. Curran Associates, Inc.

[47] Xiaoyun Xu, Zhuoran Liu, Stefanos Koffas, and Stjepan Picek. 2025. Towards Backdoor Stealthiness in Model Parameter Space. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*. Association for Computing Machinery, 2863–2876.

[48] Yi Yuan, Ruijun Deng, Zhihui Lu, and Patrick C.K. Hung. 2025. Industrial large models as carriers for malicious payloads: A fast and robust approach. *Applied Soft Computing* 185 (2025), 113967.

[49] Binchi Zhang, Zaiyi Zheng, Zhengzhang Chen, and Jundong Li. 2025. Beyond the Permutation Symmetry of Transformers: The Role of Rotation for Model Fusion. In *Proceedings of the 42nd International Conference on Machine Learning*. PMLR, 77090–77106.

[50] Shenyi Zhang, Yuchen Zhai, Keyan Guo, Hongxin Hu, Shengnan Guo, Zheng Fang, Lingchen Zhao, Chao Shen, Cong Wang, and Qian Wang. 2025. JBShield: Defending Large Language Models from Jailbreak Attacks through Activated Concept Analysis and Manipulation. In *Proceedings of the 34th USENIX Security Symposium*. USENIX Association, 8215–8234.

[51] Bo Zhao, Nima Dehmamy, Robin Walters, and Rose Yu. 2025. Understanding Mode Connectivity via Parameter Space Symmetry. In *Proceedings of the 42nd International Conference on Machine Learning*. PMLR, 77441–77460.

[52] Bo Zhao, Robin Walters, and Rose Yu. 2026. Symmetry in Neural Network Parameter Spaces. *Transactions on Machine Learning Research* (2026).

[53] Haiyan Zhao, Hanjie Chen, Fan Yang, Ninghao Liu, Huiqi Deng, Hengyi Cai, Shuaiqiang Wang, Dawei Yin, and Mengnan Du. 2024. Explainability for Large Language Models: A Survey. *ACM Trans. Intell. Syst. Technol.* 15, 2, Article 20 (2024), 38 pages.

[54] Jian Zhao, Shenao Wang, Yanjie Zhao, Xinyi Hou, Kailong Wang, Peiming Gao, Yuanchao Zhang, Chen Wei, and Haoyu Wang. 2024. Models Are Codes: Towards Measuring Malicious Code Poisoning Attacks on Pre-trained Model Hubs. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. Association for Computing Machinery, 2087–2098.

[55] Na Zhao, Kejiang Chen, Chuan Qin, Yi Yin, Weiming Zhang, and Nenghai Yu. 2023. Calibration-based Steganalysis for Neural Network Steganography. In *Proceedings of the 2023 ACM Workshop on Information Hiding and Multimedia Security*. Association for Computing Machinery, 91–96.

[56] Xiaoyan Zhao, Juntao You, Yang Zhang, Wenjie Wang, Hong Cheng, Fuli Feng, See-Kiong Ng, and Tat-Seng Chua. 2026. NextQuill: Causal Preference Modeling for Enhancing LLM Personalization. In *Proceedings of the 14th International Conference on Learning Representations*.

[57] Wanjun Zhong, Ruixiang Cui, Yiduo Guo, Yaobo Liang, Shuai Lu, Yanlin Wang, Amin Saied, Weizhu Chen, and Nan Duan. 2024. AGIEval: A Human-Centric Benchmark for Evaluating Foundation Models. In *Findings of the Association for Computational Linguistics: NAACL 2024*. Association for Computational Linguistics, 2299–2314.

[58] Ruofan Zhu, Ganhao Chen, Wenbo Shen, Xiaofei Xie, and Rui Chang. 2025. My Model is Malware to You: Transforming AI Models into Malware by Abusing TensorFlow APIs. In *2025 IEEE Symposium on Security and Privacy (SP)*. IEEE, 486–503.

[59] Liu Ziyin, Yizhou Xu, and Isaac L. Chuang. 2025. Remove Symmetries to Control Model Expressivity and Improve Optimization. In *Proceedings of the 13th International Conference on Learning Representations*.

[60] Yuxin Zuo, Shang Qu, Yifei Li, Zhang-Ren Chen, Xuekai Zhu, Ermo Hua, Kaiyan Zhang, Ning Ding, and Bowen Zhou. 2025. MedXpertQA: Benchmarking Expert-Level Medical Reasoning and Understanding. In *Proceedings of the 42nd International Conference on Machine Learning*. PMLR, 80961–80990.

A Proofs of Functional Equivalence for Complementary Pairs

Definition A.0.1 (RMSNorm). Let the input to an RMSNorm layer be $\mathbf{x} \in \mathbb{R}^d$, and let $\boldsymbol{\gamma} \in \mathbb{R}^d$ be the learnable scale parameter, where d is the embedding dimension. The RMSNorm operation is defined as:

$$\tilde{\mathbf{x}} = \mathrm{RMSNorm}(\mathbf{x}) = \frac{\mathbf{x}}{\sqrt{\frac{1}{d}\sum_{i=1}^{d} x_i^2 + \epsilon}} \odot \boldsymbol{\gamma}, \tag{8}$$

where $\odot$ denotes element-wise multiplication, and $\epsilon > 0$ is a small constant (e.g., 10^{-6}) that prevents division by zero.

A.1 I-O Boundary

Lemma A.1.1 (I-O permutation pair). *Let $\mathbf{P}_1 \in \mathbb{R}^{d\times d}$ be a permutation matrix, and let $\tilde{\mathbf{x}}_L = \mathrm{RMSNorm}(\mathbf{x}_L, \boldsymbol{\gamma}_{final})$. Then the transformation*

$$\mathbf{W}_{emb} \to \mathbf{P}_1\mathbf{W}_{emb}, \quad \boldsymbol{\gamma}_{final} \to \mathbf{P}_1\boldsymbol{\gamma}_{final}, \quad \mathbf{W}_{lm} \to \mathbf{W}_{lm}\mathbf{P}_1^\top,$$

together with, in every layer, the residual-stream compensation

$$\boldsymbol{\gamma}_{input/post} \to \mathbf{P}_1\boldsymbol{\gamma}_{input/post}, \mathbf{W}_{in} \to \mathbf{W}_{in}\mathbf{P}_1^\top, \mathbf{W}_{out} \to \mathbf{P}_1\mathbf{W}_{out},$$

where $\mathbf{W}_{in} \in \{\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V, \mathbf{W}_g, \mathbf{W}_u\}$ ranges over all intermediate input projections and $\mathbf{W}_{out} \in \{\mathbf{W}_O, \mathbf{W}_d\}$ over all intermediate output projections, leaves the model output $\mathbf{y}$ unchanged for all inputs $\mathbf{x}$.

Proof. We prove by induction that $\mathbf{x}'_\ell = \mathbf{P}_1\mathbf{x}_\ell$ for all $\ell \in \{0, \dots, L\}$.

Base case ($\ell = 0$): $\mathbf{x}'_0 = (\mathbf{P}_1\mathbf{W}_{emb})\mathbf{x} = \mathbf{P}_1\mathbf{x}_0$.

Inductive step: Assume $\mathbf{x}'_\ell = \mathbf{P}_1\mathbf{x}_\ell$. By Lemma A.2.1, the permuted RMSNorm parameters give $\tilde{\mathbf{x}}'_{\ell+1} = \mathbf{P}_1\tilde{\mathbf{x}}_{\ell+1}$. For each attention or FFN sub-block, the right factor $\mathbf{P}_1^\top$ on its input projections cancels the permuted input, giving $\mathbf{W}_{in}\mathbf{P}_1^\top\mathbf{P}_1\tilde{\mathbf{x}} = \mathbf{W}_{in}\tilde{\mathbf{x}}$, so the sub-block operates on unpermuted activations. The left factor $\mathbf{P}_1$ on its output projections then maps the result back into the permuted residual stream. Hence $\mathbf{x}'_{\ell+1} = \mathbf{P}_1\mathbf{x}_{\ell+1}$, completing the induction.

Thus $\tilde{\mathbf{x}}'_L = \mathbf{P}_1\tilde{\mathbf{x}}_L$ by Lemma A.2.1, and

$$\mathbf{y}' = (\mathbf{W}_{lm}\mathbf{P}_1^\top)\tilde{\mathbf{x}}'_L = \mathbf{W}_{lm}\mathbf{P}_1^\top\mathbf{P}_1\tilde{\mathbf{x}}_L = \mathbf{W}_{lm}\tilde{\mathbf{x}}_L = \mathbf{y}.$$

□

Lemma A.1.2 (I-O scaling pair). *Let $\mathbf{C}_1 \in \mathbb{R}^{d\times d}$ be a positive diagonal matrix. Then the transformation*

$$\boldsymbol{\gamma}_{final} \to \mathbf{C}_1\boldsymbol{\gamma}_{final}, \quad \mathbf{W}_{lm} \to \mathbf{W}_{lm}\mathbf{C}_1^{-1}$$

leaves the model output $\mathbf{y}$ unchanged for all inputs $\mathbf{x}$.

Proof. By the definition of RMSNorm, scaling $\boldsymbol{\gamma}_{final}$ by $\mathbf{C}_1$ scales the output by $\mathbf{C}_1$:

$$\tilde{\mathbf{x}}'_L = \mathrm{RMSNorm}(\mathbf{x}_L, \mathbf{C}_1\boldsymbol{\gamma}_{final}) = \mathbf{C}_1\,\mathrm{RMSNorm}(\mathbf{x}_L, \boldsymbol{\gamma}_{final}) = \mathbf{C}_1\tilde{\mathbf{x}}_L.$$

Hence

$$\mathbf{y}' = \mathbf{W}_{lm}\mathbf{C}_1^{-1}\tilde{\mathbf{x}}'_L = \mathbf{W}_{lm}\mathbf{C}_1^{-1}\mathbf{C}_1\tilde{\mathbf{x}}_L = \mathbf{W}_{lm}\tilde{\mathbf{x}}_L = \mathbf{y}.$$

□

A.2 Attention Boundary

Lemma A.2.1 (Permutation propagation through RMSNorm). *If $\mathbf{x}' = \mathbf{P}\mathbf{x}$ and $\boldsymbol{\gamma}' = \mathbf{P}\boldsymbol{\gamma}$, then*

$$\tilde{\mathbf{x}}' = \mathbf{P}\tilde{\mathbf{x}}.$$

Proof. Since $\sum_i (x'_i)^2 = \sum_i x_i^2$, we have

$$\tilde{\mathbf{x}}' = \frac{\mathbf{P}\mathbf{x}}{\sqrt{\frac{1}{d}\sum_i x_i^2 + \epsilon}} \odot (\mathbf{P}\boldsymbol{\gamma}) = \mathbf{P}\left(\frac{\mathbf{x}}{\sqrt{\frac{1}{d}\sum_i x_i^2 + \epsilon}} \odot \boldsymbol{\gamma}\right) = \mathbf{P}\tilde{\mathbf{x}}.$$

□

Lemma A.2.2 (Norm-to-QKV scaling pair). *Let $\mathbf{C}_2$ be a positive diagonal matrix, and let RoPE act on the output side of $\mathbf{Q}$ and $\mathbf{K}$ as $\mathbf{Q}\mathbf{R}_m$, where $\mathbf{R}_m$ depends only on position m. Then the joint transformation*

$$\boldsymbol{\gamma}_{input} \to \mathbf{C}_2\boldsymbol{\gamma}_{input}, \mathbf{W}_Q \to \mathbf{W}_Q\mathbf{C}_2^{-1}, \mathbf{W}_K \to \mathbf{W}_K\mathbf{C}_2^{-1}, \mathbf{W}_V \to \mathbf{W}_V\mathbf{C}_2^{-1}$$

leaves the queries, keys, values, and the attention output unchanged.

Proof. By Lemma A.1.2, the scaled normalization output is $\tilde{\mathbf{x}}' = \mathbf{C}_2\tilde{\mathbf{x}}$. Hence

$$\mathbf{Q}' = \mathbf{W}_Q\mathbf{C}_2^{-1}\tilde{\mathbf{x}}' = \mathbf{W}_Q\mathbf{C}_2^{-1}\mathbf{C}_2\tilde{\mathbf{x}} = \mathbf{W}_Q\tilde{\mathbf{x}} = \mathbf{Q},$$

and similarly $\mathbf{K}' = \mathbf{K}$ and $\mathbf{V}' = \mathbf{V}$.

Since the diagonal scaling cancels exactly before any rotation is applied, for each head h,

$$(\mathbf{Q}'_h\mathbf{R}_m) = (\mathbf{W}_{Q,h}\mathbf{C}_2^{-1}\tilde{\mathbf{x}}')\mathbf{R}_m = \mathbf{W}_{Q,h}\tilde{\mathbf{x}}\mathbf{R}_m = \mathbf{Q}_h\mathbf{R}_m,$$

so the rotary-encoded queries and keys, and therefore the attention scores, are preserved. Since $\mathbf{V}$ is unchanged, the attention output is invariant. □

Lemma A.2.3 (Head-permutation pair). *Let $\mathbf{P}_2$ be a head-level permutation matrix that permutes the H_a heads as whole blocks of d_h rows. Then the joint transformation*

$$\mathbf{W}_Q \to \mathbf{P}_2\mathbf{W}_Q, \mathbf{W}_K \to \mathbf{P}_2\mathbf{W}_K, \mathbf{W}_V \to \mathbf{P}_2\mathbf{W}_V, \mathbf{W}_O \to \mathbf{W}_O\mathbf{P}_2^\top$$

leaves the attention output invariant. For Grouped Query Attention (GQA), $\mathbf{P}_2$ is restricted to permute KV-head groups jointly.

Proof. Since RoPE acts on the within-head dimension d_h via $\mathbf{Q}\mathbf{R}_m$, while $\mathbf{P}_2$ acts on the across-heads dimension, the two operations commute:

$$\mathbf{P}_2(\mathbf{Q}\mathbf{R}_m) = (\mathbf{P}_2\mathbf{Q})\mathbf{R}_m.$$

Thus the transformed rotary-encoded queries and keys satisfy $\tilde{\mathbf{Q}}' = \mathbf{P}_2\tilde{\mathbf{Q}}$ and $\tilde{\mathbf{K}}' = \mathbf{P}_2\tilde{\mathbf{K}}$, and $\mathbf{V}' = \mathbf{P}_2\mathbf{V}$.

Attention scores are preserved because

$$(\tilde{\mathbf{Q}}'_i)^\top\tilde{\mathbf{K}}'_j = (\mathbf{P}_2\tilde{\mathbf{Q}}_i)^\top(\mathbf{P}_2\tilde{\mathbf{K}}_j) = \tilde{\mathbf{Q}}_i^\top\tilde{\mathbf{K}}_j,$$

so $a'_{ij} = a_{ij}$. The attention output becomes

$$\mathbf{Z}'_i = \sum_j a'_{ij}\mathbf{V}'_j = \sum_j a_{ij}\mathbf{P}_2\mathbf{V}_j = \mathbf{P}_2\mathbf{Z}_i.$$

Finally,

$$\mathbf{O}' = \mathbf{W}_O\mathbf{P}_2^\top\mathbf{Z}' = \mathbf{W}_O\mathbf{P}_2^\top\mathbf{P}_2\mathbf{Z} = \mathbf{W}_O\mathbf{Z} = \mathbf{O}.$$

For GQA, $\mathbf{P}_2$ permutes whole KV-groups, each containing one K-head, one V-head, and the associated Q-heads, so the sharing structure is preserved and the output remains invariant. □

Lemma A.2.4 (Value-Output scaling pair)**.** *Let* $\mathbf{C}_3$ *be a positive diagonal matrix, constant within each head's block of* d_h *rows (and within each KV-group for GQA). Then*

$$\mathbf{W}_V \to \mathbf{C}_3\mathbf{W}_V, \quad \mathbf{W}_O \to \mathbf{W}_O\mathbf{C}_3^{-1}$$

preserves the attention output $\mathbf{O}$*.*

Proof. Since $\mathbf{W}_Q$ and $\mathbf{W}_K$ are unchanged, attention scores remain invariant. The value projection becomes $\mathbf{V}' = \mathbf{C}_3\mathbf{V}$, so

$$\mathbf{Z}'_i = \sum_j a'_{ij}\mathbf{V}'_j = \sum_j a_{ij}\mathbf{C}_3\mathbf{V}_j = \mathbf{C}_3\mathbf{Z}_i.$$

Then the output projection yields

$$\mathbf{O}' = \mathbf{W}_O\mathbf{C}_3^{-1}\mathbf{Z}' = \mathbf{W}_O\mathbf{C}_3^{-1}\mathbf{C}_3\mathbf{Z} = \mathbf{W}_O\mathbf{Z} = \mathbf{O}.$$

For GQA, $\mathbf{C}_3$ is constant across each KV-group, so the inverse applies consistently to $\mathbf{W}_O$. □

A.3 FFN Boundary

Lemma A.3.1 (Norm-to-FFN scaling pair)**.** *Let* $\mathbf{C}_4$ *be a positive diagonal matrix, and let the FFN compute* $\mathbf{h} = \sigma(\mathbf{W}_g\tilde{\mathbf{x}}) \odot (\mathbf{W}_u\tilde{\mathbf{x}})$ *with element-wise SiLU* σ*. Then*

$$\boldsymbol{\gamma}_{post} \to \mathbf{C}_4\boldsymbol{\gamma}_{post}, \quad \mathbf{W}_g \to \mathbf{W}_g\mathbf{C}_4^{-1}, \quad \mathbf{W}_u \to \mathbf{W}_u\mathbf{C}_4^{-1}$$

leaves $\mathbf{h}$ *invariant.*

Proof. By Lemma A.1.2, $\tilde{\mathbf{x}}' = \mathbf{C}_4\tilde{\mathbf{x}}$. Hence

$$\mathbf{a}' = \mathbf{W}_g\mathbf{C}_4^{-1}\tilde{\mathbf{x}}' = \mathbf{W}_g\tilde{\mathbf{x}} = \mathbf{a},$$

and similarly $\mathbf{u}' = \mathbf{u}$. Since σ is element-wise, $\sigma(\mathbf{a}') = \sigma(\mathbf{a})$, so

$$\mathbf{h}' = \sigma(\mathbf{a}') \odot \mathbf{u}' = \sigma(\mathbf{a}) \odot \mathbf{u} = \mathbf{h}.$$

□

Lemma A.3.2 (FFN neuron permutation pair)**.** *Let* $\mathbf{P}_3$ *be a permutation matrix. Then*

$$\mathbf{W}_g \to \mathbf{P}_3\mathbf{W}_g, \quad \mathbf{W}_u \to \mathbf{P}_3\mathbf{W}_u, \quad \mathbf{W}_d \to \mathbf{W}_d\mathbf{P}_3^\top$$

leaves the FFN output invariant.

Proof. The permuted gate and up projections give $\mathbf{a}' = \mathbf{P}_3\mathbf{a}$ and $\mathbf{u}' = \mathbf{P}_3\mathbf{u}$. Since SiLU is element-wise, it commutes with permutations, so

$$\mathbf{h}' = \sigma(\mathbf{a}') \odot \mathbf{u}' = \mathbf{P}_3\sigma(\mathbf{a}) \odot \mathbf{P}_3\mathbf{u} = \mathbf{P}_3\mathbf{h}.$$

The down projection then cancels this permutation: $\mathbf{W}_d\mathbf{P}_3^\top\mathbf{h}' = \mathbf{W}_d\mathbf{h}$. □

Lemma A.3.3 (Up-Down scaling pair)**.** *Let* $\mathbf{C}_5$ *be a positive diagonal matrix. Then*

$$\mathbf{W}_u \to \mathbf{C}_5\mathbf{W}_u, \quad \mathbf{W}_d \to \mathbf{W}_d\mathbf{C}_5^{-1}$$

preserves the FFN output.

Proof. Since $\mathbf{W}_g$ is unchanged, $\mathbf{a}$ and $\sigma(\mathbf{a})$ are invariant. The up projection gives $\mathbf{u}' = \mathbf{C}_5\mathbf{u}$, so

$$\mathbf{h}' = \sigma(\mathbf{a}') \odot \mathbf{u}' = \sigma(\mathbf{a}) \odot \mathbf{C}_5\mathbf{u} = \mathbf{C}_5\mathbf{h}.$$

The down projection then cancels the scaling:

$$\mathbf{y}' = \mathbf{W}_d\mathbf{C}_5^{-1}\mathbf{h}' = \mathbf{W}_d\mathbf{C}_5^{-1}\mathbf{C}_5\mathbf{h} = \mathbf{W}_d\mathbf{h} = \mathbf{y}.$$

□

A.4 Additional Architectures

Lemma A.4.1 (Multimodal models)**.** *Let* $\mathbf{W}_{img}$ *and* $\mathbf{W}_{img}^{out}$ *be the image embedding and output projections. The joint transformation*

$$\mathbf{W}_{img} \to \mathbf{P}_1\mathbf{W}_{img}, \qquad \mathbf{W}_{img}^{out} \to \mathbf{W}_{img}^{out}\mathbf{P}_1^\top,$$

together with the layer-wise propagation of $\mathbf{P}_1$ *(Lemma A.1.1), preserves the model output. As with* $\mathbf{W}_{emb}$*, no scaling applies to* $\mathbf{W}_{img}$ *alone. Cross-modal attention is structurally identical to standard attention, so Lemmas A.2.2–A.2.4 apply unchanged.*

Proof. Image tokens enter the residual stream exactly like text tokens via $\mathbf{W}_{img}$, so Lemma A.1.1 extends verbatim. Cross-modal attention uses the same hidden dimension d, and the arguments of Lemmas A.2.2–A.2.4 are modality-independent. Hence Lemmas A.1.1–A.3.3 preserve functional equivalence. □

Lemma A.4.2 (Mixture-of-Experts (MoE))**.** *In an MoE layer, the router and all experts read the same normalized input* $\tilde{\mathbf{x}}$*. Applying the expert transformations of Lemmas A.3.1–A.3.3 preserves the MoE output if the router weights are also compensated by* $\mathbf{P}_1^\top\mathbf{C}_4^{-1}$ *under the residual-stream permutation and norm scaling, or by a permutation matrix* Π *when the experts are permuted by* Π*.*

Proof. Expert outputs are unchanged by Lemmas A.3.1–A.3.3. Since the router input becomes $\tilde{\mathbf{x}}' = \mathbf{C}_4\mathbf{P}_1\tilde{\mathbf{x}}$ in the composed model, compensating the router weights by $\mathbf{P}_1^\top\mathbf{C}_4^{-1}$ gives $\mathbf{g}' = \mathbf{g}$. If the experts are permuted by Π and the router weights are also left-multiplied by Π, then $\mathbf{g}' = \Pi\mathbf{g}$ merely re-indexes the unchanged sum, so $\mathbf{y}'_{moe} = \mathbf{y}_{moe}$. □

B Ethical Considerations

This work investigates the security vulnerabilities of open-source LLMs and evaluates malware embedding attacks to verify the effectiveness of our defense. All evaluated attack methods, malware samples, models, and datasets are publicly available and involve no privacy issues. We adhere to responsible disclosure principles and will make the defense artifacts available to the research community upon publication. We acknowledge the dual-use risk of publishing PAA and JAA in reproducible detail. However, this risk is mitigated by the severe capacity constraints of both attacks, and further defeated by SymSan$^+$ (Section 5.2.3).

C Open Science

The source code is available at https://github.com/uuutmtm/SymSan.

D Acknowledgements

This work was supported in part by the National Key Research and Development Program of China (No. 2023YFB2906400; No. 2024YFB2907202), in part by the National Natural Science Foundation of China (No. 62502366), in part by the Postdoctoral Fellowship Program of CPSF (Grant Number GZB20250803; 2025M781514), and in part by the Shaanxi Postdoctoral Research Foundation (No. 2025BSHSDZZ204). This paper was edited for grammar using Qwen, Deepseek, and GLM.[4]

[4]https://chat.qwen.ai/; https://chat.deepseek.com/; https://chat.z.ai/